

Ульяновский государственный университет
Факультет математики информационных и авиационных технологий
Кафедра информационных технологий

Лабораторная работа № 1

Метрическая оценка программного обеспечения

Вариант № 6
Каталог автомобилей

Выполнил
студент группы ПРИ-О-25/1
Ермолаев Е.А.
Проверил
Волков М.А.

Ульяновск 2026

Содержание

- 1 Задание и состав программ
- 2 Метрики размера программ
- 3 Метрики сложности потока управления
- 4 Метрики сложности потока данных
- 5 Метрики стилистики и технологических средств
- 6 Сравнение результатов
- Приложение А Словари операторов и операндов
- Приложение Б Классификация переменных и спен
- Приложение В Подграфы для метрики граничных значений
- Приложение Г Блок схемы и графы управления
- Приложение Д Листинги программ

1 Задание и состав программ

Цель работы — разработать каталог автомобилей тремя способами программирования и сравнить полученные реализации по метрикам размера, потока управления и потока данных.

Разработаны линейная программа, программа с динамической переменной типа указатель и модульная программа. Каждая версия добавляет записи в бинарный файл cars.dat, выводит все записи, ищет по каждому атрибуту и завершает работу по пункту меню. Строковый поиск выполнен по совпадению подстроки с учётом регистра; числовой поиск — по включительному интервалу.

Поле	Тип	Содержание
brand	char[30]	Марка автомобиля
model	char[30]	Модель
year	int	Год выпуска от 1886 до 2026
price	int	Положительная цена
body	char[20]	Тип кузова
segment	char[10]	Сегмент автомобиля

Версия	Исходные файлы
Линейная	lab1/linear.cpp
С указателями	lab1/pointers.cpp
Модульная	modular/main.cpp, modular/car.h, modular/functions.h

Расчёт выполнен для исходников репозитория qvenx/lab-software-development-methods, коммит d8ae75f от 8 октября 2026 года. Заголовочные файлы учитываются один раз. Реализации модульных функций находятся в functions.h.

2 Метрики размера программ

2 1 Метрика Холстеда

Словари операторов и операндов с частотами приведены в приложении А.

Операторы: управляющие конструкции, имена функций, операции и разделители. Операнды: идентификаторы данных и литералы. Литерал считается одним операндом; каждая скобка — одним вхождением, пары объединены в словаре.

Директивы препроцессора, библиотечный код, char, int, void, const, struct, using, namespace, std, cin, cout, endl, left, ios, binary, app, numeric_limits и streamsize исключены. Car, типы файловых потоков и поля структуры учитываются как идентификаторы.

Показатель	Линейная	С указателями	Модульная
n_1 уникальных операторов	46	49	57
n_2 уникальных операндов	100	100	100
N_1 всего операторов	1988	1985	1708
N_2 всего операндов	551	553	380
n словарь программы	146	149	157
N длина программы	2539	2538	2088

$$n = n_1 + n_2; N = N_1 + N_2; \hat{N} = n_1 \log_2 n_1 + n_2 \log_2 n_2; V = N \log_2 n.$$

Потенциальный объём: $V^* = n^* \log_2 n^* = 19 \log_2 19 = 80,710623$. Принята общая модель интерфейса: $n_1^* = 2, n_2^* = 17, n^* = 19$. В 17 операндов входят шесть полей записи, два выбора меню, четыре строковых запроса, четыре числовые границы и файл данных.

$$L = \frac{V^*}{V}; \hat{L} = \frac{2n_2}{n_1 N_2}; I = \hat{L} V.$$

Усилия: $E_{\text{теор}} = \frac{\hat{N} \log_2 n}{L}; E_{\text{факт}} = \frac{V}{L} = \frac{V^2}{V^*}$. Все логарифмы имеют основание 2; округляются конечные результаты.

Показатель	Линейная	С указателями	Модульная
$\hat{N}$ расчётная длина	918,469469	939,506401	996,860350
V объём	18254,964555	18322,249705	15231,168124
V^* потенциальный объём	80,710623	80,710623	80,710623
L уровень программы	0,004421	0,004405	0,005299
$\hat{L}$ оценка уровня	0,007891	0,007381	0,009234
I содержание	144,046118	135,234526	140,638672
Ефакт усилия	4128870,767284	4159363,696984	2874324,029378

Показатель	Линейная	С указателями	Модульная
Етеор	1493596,589608	1539696,146105	1372269,950824

3 Метрики сложности потока управления

3 1 Метрика Маккейба

$G = e - v + 2p$, где e — дуги, v — вершины, p — связные компоненты. Для каждой функции $p = 1$. Последовательные действия объединены в базовые блоки; составное условие — одна вершина отбора. Учитываются все ветви case и default.

Версия и функция	v	e	p	G
Линейная / main	145	200	1	57
С указателями / main	145	200	1	57
Модульная / main	14	19	1	7
Модульная / addCar	36	46	1	12
Модульная / showAll	9	10	1	3
Модульная / findCar	12	19	1	9
Модульная / searchByBrand	16	20	1	6
Модульная / searchByModel	16	20	1	6
Модульная / searchByYear	16	20	1	6
Модульная / searchByPrice	16	20	1	6
Модульная / searchByBody	16	20	1	6
Модульная / searchBySegment	16	20	1	6
Модульная / printHeader	3	2	1	1
Модульная / printCar	3	2	1	1

Линейная и указательная версии: $G = 200 - 145 + 2 = 57$. Для модульной версии: $\max G_i = 12$,

$$\sum_{i=1}^{12} G_i = 69. \text{ С общей базовой единицей: } 1 + \sum_{i=1}^{12} (G_i - 1) = 58.$$

Графы с нумерацией вершин приведены в приложении Г. Для модульной версии каждая функция имеет отдельный граф.

3 2 Метрика граничных значений

Подграф вершины отбора включает саму вершину и заканчивается перед ближайшей общей нижней границей. Его сложность равна числу вершин. Принимающая вершина с одной исходящей дугой имеет сложность 1; конечная — 0.

$S_a = \sum_j s_j + r$; $S_0 = 1 - \frac{v-1}{S_a}$, где s_j — сложности подграфов вершин отбора, r — число принимающих вершин с одной дугой. Подграфы приведены в приложении В.

Версия и функция	Sa	S ₀
Линейная / main	622	0,768489
С указателями / main	622	0,768489
Модульная / main	28	0,535714
Модульная / addCar	63	0,444444
Модульная / showAll	14	0,428571
Модульная / findCar	18	0,388889

Версия и функция	S _a	S ₀
Модульная / searchByBrand	30	0,500000
Модульная / searchByModel	30	0,500000
Модульная / searchByYear	26	0,423077
Модульная / searchByPrice	26	0,423077
Модульная / searchByBody	30	0,500000
Модульная / searchBySegment	30	0,500000
Модульная / printHeader	2	0
Модульная / printCar	2	0

Линейная и указательная версии: $S_a = 531 + 91 = 622$; $S_0 = 1 - \frac{144}{622} = 0,768489$. Для модульной версии показатели рассчитаны по функциям.

3.3 Метрика Джилба

CL — число if и switch; $cl = \frac{CL}{N_1}$. CLI — максимальная вложенность управляющих конструкций, включая циклы. Внешний уровень — первый; else относится к своему if. Вложенность вызванных функций не прибавляется.

Показатель	Линейная	С указателями	Модульная
CL число условий	29	29	24
cl относительная сложность	0,014588	0,014610	0,014052
CLI вложенность	6	6	3

Версия и функция	CL	CLI
Линейная / main	29	6
С указателями / main	29	6
Модульная / main	1	2
Модульная / addCar	5	2
Модульная / showAll	1	2
Модульная / findCar	1	1
Модульная / searchByBrand	3	3
Модульная / searchByModel	3	3
Модульная / searchByYear	2	3
Модульная / searchByPrice	2	3
Модульная / searchByBody	3	3
Модульная / searchBySegment	3	3
Модульная / printHeader	0	0
Модульная / printCar	0	0

4 Метрики сложности потока данных

4 1 Спен

$span(x) = k_x - 1$, где k_x — число появлений переменной, включая объявление. Учитываются локальные переменные и параметры в своих областях видимости. `car` считается целиком; поля, функции, типы, литералы и стандартные потоки отдельно не учитываются.

Показатель	Линейная	С указателями	Модульная
Переменных и параметров	19	19	27
Суммарный спен	148	149	113

4 2 Метрика Чепина

$Q = P + 2M + 3C + 0,5T$. P — исходные данные для обработки и вывода; M — создаваемые или изменяемые данные; C — данные в условиях управления; T — неиспользуемые переменные. Группы могут пересекаться.

В P входят `car` и параметры поиска; в M — локальные переменные; в C — переменные условий проверки ввода, поиска, меню и файловых потоков. Параметр `const Car& car` функции `printCar` входит только в P . Классификация приведена в приложении Б.

Показатель	Линейная	С указателями	Модульная
P	9	9	17
M	19	19	26
C	19	19	26
T	0	0	0
Q метрика Чепина	104	104	147

4 3 Пары модуль и глобальная переменная

$R_{up} = \frac{A_{up}}{P_{up}}$, где A_{up} — фактические пары «модуль — глобальная переменная», P_{up} — возможные.

Пользовательских глобальных переменных нет: $A_{up} = P_{up} = 0$; R_{up} не определено.

Поток данных: ввод и проверка → структура `Car` → `cars.dat`; чтение из файла → проверка критерия поиска → вывод. В модульной версии запись передаётся в `printCar` через `const Car&`.

5 Метрики стилистики и технологических средств

5 1 Уровень комментированности

$F = \frac{N_{\text{ком}}}{N_{\text{стр}}} = 0$: комментариев нет. Учитываются все физические строки. Тексты разделены на шесть последовательных частей максимально равной длины.

Показатель	Линейная	С указателями	Модульная
Физических строк	509	533	524
Комментариев	0	0	0

$F_i = \text{sign}\left(\frac{N_{\text{ком},i}}{N_{\text{стр},i}} - 0,1\right) = -1; \sum_{i=1}^6 F_i = -6$. Нормальный уровень комментированности не достигнут.

Версия	Строки в сегментах	ΣF_i
Линейная	85, 85, 85, 85, 85, 84	-6
С указателями	89, 89, 89, 89, 89, 88	-6
Модульная	88, 88, 87, 87, 87, 87	-6

5 2 Метрики Денисенко

$$g_{\text{тип}} = T \cdot \frac{T(T-1) + N_1(n_1-1) + N_2(n_2-1)}{N_1(n_1-T) + N_2(n_2-T)}.$$

$T=6$: char, char[], int, Car (Car*), ifstream, ofstream. Ссылки относятся к типу Car. Область доступности:

$$g_m = \frac{T(T-1) + N_1(n_1-1) + N_2(n_2-1) + N_2 n_{\text{св}}}{T(T-1) + N_1(n_1-1) + N_2(n_2-1)}.$$

Связей через глобальные переменные нет: $n_{\text{св}}=0$; $g_m=1$.

Показатель	Линейная	С указателями	Модульная
gтип контроль типов	6,581431	6,555713	6,511447
gm область доступности	1	1	1

5 3 Уровень языка программирования

$$\lambda = \hat{L}^2 V; \bar{\lambda} = \frac{\lambda_1 + \lambda_2 + \lambda_3}{3}; \sigma = \sqrt{\frac{\sum_{i=1}^3 (\lambda_i - \bar{\lambda})^2}{3}}.$$

Показатель	Линейная	С указателями	Модульная
λ уровень языка	1,136638	0,998151	1,298603

$\bar{\lambda}=1,144464$; $\sigma=0,122784$.

6 Сравнение результатов

Модульная версия имеет меньшие длину и объём по Холстеду и суммарный спен. Максимальная сложность функции снизилась с 57 до 12, вложенность — с 6 до 3. Линейная и указательная версии имеют одинаковую топологию графа.

Приложение А Словари операторов и операндов

Линейная

Оператор	Частота	Операнд	Частота
!	16	" to 2026: "	1
!=	5	"0 - Exit"	1
&	8	"0 - back"	1
&&	2	"1 - Add car"	1
()	594	"1 - brand"	1
*	8	"2 - Show all"	1
,	65	"2 - model"	1
.	153	"3 - Search"	1
:	5	"3 - year"	1
::	31	"4 - price"	1
;	179	"5 - body"	1
<	39	"6 - segment "	1
<<	322	"Body cannot be empty"	1
<=	3	"Body car cannot be empty"	1
==	14	"Body car is too long. Maximum 19 characters"	1
>	35	"Body is too long. Maximum characters 19"	1
>=	2	"Body"	7
>>	8	"Body: "	1
[]	16	"Brand cannot be empty"	2
break	5	"Brand is too long. Maximum 29 characters"	1
case	4	"Brand is too long. Maximum 29 characters: "	1
clear	16	"Brand"	7
close	8	"Brand: "	1
default	1	"Car added."	1
do	1	"Enter Brand: "	1
else	16	"Enter a number: "	1
fail	16	"Enter body: "	1
getline	16	"Enter model: "	1
if	28	"Enter segment: "	1
ignore	22	"Error file read"	1
main	1	"Exit"	1
max	22	"File open error"	7
read	7	"From price"	1
reinterpret_cast	8	"From year: "	1
return	1	"Model cannot be empty"	2
setw	84	"Model is too long. Maximum 29 characters"	1
sizeof	8	"Model is too long. Maximum characters 29"	1
string	7	"Model"	7
strlen	8	"Model: "	1
strstr	4	"Price"	7
switch	1	"Price: "	1
while	24	"Segment cannot be empty"	2

Оператор	Частота	Операнд	Частота
write	1	"Segment is too long. Maximum 9 characters"	1
{}	150	"Segment is too long. Maximum characters 9"	1
	1	"Segment"	7
	23	"Segment: "	1
		"To price"	1
		"To year: "	1
		"Wrong choice. Enter 0-6: "	1
		"Wrong choise"	1
		"Wrong price. Enter correct price."	2
		"Wrong price. Enter positive number: "	1
		"Wrong year. Enter year from "	1
		"Wrong year. Enter year from 1886 to 2026"	2
		"Year"	7
		"Year: "	1
		"cars.dat"	8
		"search by: "	1
		'_'	7
		'\n'	22
		0	14
		1	4
		10	6
		11	14
		116	7
		14	14
		1886	3
		2	2
		20	6
		2026	3
		21	14
		3	3
		30	12
		31	28
		4	1
		5	1
		6	2
		8	14
		Car	10
		body	12
		brand	12

Оператор	Частота	Операнд	Частота
		car	76
		choice	6
		file	32
		ifstream	7
		maxPrice	5
		maxYear	6
		minPrice	5
		minYear	7
		model	12
		nullptr	4
		ofstream	1
		price	12
		searchBody	5
		searchBrand	5
		searchChoice	10
		searchModel	5
		searchSegment	5
		segment	12
		year	13

$n_1 = 46 ; N_1 = 1988 ; n_2 = 100 ; N_2 = 551$.

С указателями

Оператор	Частота	Операнд	Частота
!	16	" to 2026: "	1
!=	5	"0 - Exit"	1
&&	2	"0 - back"	1
()	594	"1 - Add car"	1
*	9	"1 - brand"	1
,	65	"2 - Show all"	1
->	67	"2 - model"	1
.	86	"3 - Search"	1
:	5	"3 - year"	1
::	31	"4 - price"	1
;	180	"5 - body"	1
<	39	"6 - segment "	1
<<	322	"Body cannot be empty"	1
<=	3	"Body car cannot be empty"	1
=	1	"Body car is too long. Maximum 19 characters"	1
==	14	"Body is too long. Maximum characters 19"	1
>	35	"Body"	7
>=	2	"Body: "	1
>>	8	"Brand cannot be empty"	2
[]	16	"Brand is too long. Maximum 29 characters"	1
break	5	"Brand is too long. Maximum 29 characters: "	1
case	4	"Brand"	7
clear	16	"Brand: "	1
close	8	"Car added."	1
default	1	"Enter Brand: "	1
delete	1	"Enter a number: "	1
do	1	"Enter body: "	1
else	16	"Enter model: "	1
fail	16	"Enter segment: "	1
getline	16	"Error file read"	1
if	28	"Exit"	1
ignore	22	"File open error"	7
main	1	"From price"	1
max	22	"From year: "	1
new	1	"Model cannot be empty"	2
read	7	"Model is too long. Maximum 29 characters"	1
reinterpret_cast	8	"Model is too long. Maximum characters 29"	1
return	1	"Model"	7
setw	84	"Model: "	1
sizeof	8	"Price"	7
string	7	"Price: "	1
strlen	8	"Segment cannot be empty"	2
strstr	4	"Segment is too long. Maximum 9 characters"	1
switch	1	"Segment is too long. Maximum characters 9"	1

Оператор	Частота	Операнд	Частота
while	24	"Segment"	7
write	1	"Segment: "	1
{}	150	"To price"	1
	1	"To year: "	1
	23	"Wrong choice. Enter 0-6: "	1
		"Wrong choise"	1
		"Wrong price. Enter correct price."	2
		"Wrong price. Enter positive number: "	1
		"Wrong year. Enter year from "	1
		"Wrong year. Enter year from 1886 to 2026"	2
		"Year"	7
		"Year: "	1
		"cars.dat"	8
		"search by: "	1
		'.'	7
		'\n'	22
		0	14
		1	4
		10	6
		11	14
		116	7
		14	14
		1886	3
		2	2
		20	6
		2026	3
		21	14
		3	3
		30	12
		31	28
		4	1
		5	1
		6	2
		8	14
		Car	11
		body	12
		brand	12
		car	77
		choice	6

Оператор	Частота	Операнд	Частота
		file	32
		ifstream	7
		maxPrice	5
		maxYear	6
		minPrice	5
		minYear	7
		model	12
		nullptr	4
		ofstream	1
		price	12
		searchBody	5
		searchBrand	5
		searchChoice	10
		searchModel	5
		searchSegment	5
		segment	12
		year	13

$n_1 = 49; N_1 = 1985; n_2 = 100; N_2 = 553.$

Модульная

Оператор	Частота	Операнд	Частота
!	16	" to 2026: "	1
!=	5	"0 - Exit"	1
&	10	"0 - back"	1
&&	2	"1 - Add car"	1
()	518	"1 - brand"	1
*	8	"2 - Show all"	1
,	59	"2 - model"	1
.	117	"3 - Search"	1
:	13	"3 - year"	1
::	31	"4 - price"	1
;	211	"5 - body"	1
<	39	"6 - segment "	1
<<	142	"Body cannot be empty"	1
<=	3	"Body car cannot be empty"	1
==	8	"Body car is too long. Maximum 19 characters"	1
>	35	"Body is too long. Maximum characters 19"	1
>=	2	"Body"	1
>>	8	"Body: "	1
[]	16	"Brand cannot be empty"	2
addCar	3	"Brand is too long. Maximum 29 characters"	1
break	13	"Brand is too long. Maximum 29 characters: "	1
case	11	"Brand"	1
clear	16	"Brand: "	1
close	8	"Car added."	1
default	2	"Enter Brand: "	1
do	1	"Enter a number: "	1
else	16	"Enter body: "	1
fail	16	"Enter model: "	1
findCar	3	"Enter segment: "	1
getline	16	"Error file read"	1
if	22	"Exit"	1
ignore	22	"File open error"	7
main	1	"From price"	1
max	22	"From year: "	1
printCar	9	"Model cannot be empty"	2
printHeader	9	"Model is too long. Maximum 29 characters"	1
read	7	"Model is too long. Maximum characters 29"	1
reinterpret_cast	8	"Model"	1
return	1	"Model: "	1
searchByBody	3	"Price"	1
searchByBrand	3	"Price: "	1
searchByModel	3	"Segment cannot be empty"	2
searchByPrice	3	"Segment is too long. Maximum 9 characters"	1
searchBySegment	3	"Segment is too long. Maximum characters 9"	1

Оператор	Частота	Операнд	Частота
searchByYear	3	"Segment"	1
setw	12	"Segment: "	1
showAll	3	"To price"	1
sizeof	8	"To year: "	1
string	1	"Wrong choice"	1
strlen	8	"Wrong choice. Enter 0-6: "	1
strstr	4	"Wrong price. Enter correct price."	2
switch	2	"Wrong price. Enter positive number: "	1
while	24	"Wrong year. Enter year from "	1
write	1	"Wrong year. Enter year from 1886 to 2026"	2
{ }	154	"Year"	1
	1	"Year: "	1
	23	"cars.dat"	8
		"search by: "	1
		'_'	1
		'\n'	22
		0	15
		1	4
		10	6
		11	2
		116	1
		14	2
		1886	3
		2	2
		20	6
		2026	3
		21	2
		3	3
		30	12
		31	4
		4	1
		5	1
		6	2
		8	2
		Car	19
		body	6
		brand	6
		car	56
		choice	6
		file	32
		ifstream	7

Оператор	Частота	Операнд	Частота
		maxPrice	5
		maxYear	6
		minPrice	5
		minYear	7
		model	6
		nullptr	4
		ofstream	1
		price	6
		searchBody	5
		searchBrand	5
		searchChoice	5
		searchModel	5
		searchSegment	5
		segment	6
		year	7

$n_1=57; N_1=1708; n_2=100; N_2=380.$

Приложение Б Классификация переменных и спен

Линейная

Функция	Переменная	k	k – 1	P	M	C	T
main	car #1	76	75	1	1	1	0
main	choice #2	6	5	0	1	1	0
main	file #3	4	3	0	1	1	0
main	file #4	4	3	0	1	1	0
main	searchChoice #5	10	9	0	1	1	0
main	searchBrand #6	5	4	1	1	1	0
main	file #7	4	3	0	1	1	0
main	searchModel #8	5	4	1	1	1	0
main	file #9	4	3	0	1	1	0
main	minYear #10	7	6	1	1	1	0
main	maxYear #11	6	5	1	1	1	0
main	file #12	4	3	0	1	1	0
main	minPrice #13	5	4	1	1	1	0
main	maxPrice #14	5	4	1	1	1	0
main	file #15	4	3	0	1	1	0
main	searchBody #16	5	4	1	1	1	0
main	file #17	4	3	0	1	1	0
main	searchSegment #18	5	4	1	1	1	0
main	file #19	4	3	0	1	1	0

$\sum_x span(x) = 148; Q = 104$. Номер после имени различает объявления в разных областях видимости.

С указателями

Функция	Переменная	k	k – 1	P	M	C	T
main	car #1	77	76	1	1	1	0
main	choice #2	6	5	0	1	1	0
main	file #3	4	3	0	1	1	0
main	file #4	4	3	0	1	1	0
main	searchChoice #5	10	9	0	1	1	0
main	searchBrand #6	5	4	1	1	1	0
main	file #7	4	3	0	1	1	0
main	searchModel #8	5	4	1	1	1	0
main	file #9	4	3	0	1	1	0
main	minYear #10	7	6	1	1	1	0
main	maxYear #11	6	5	1	1	1	0
main	file #12	4	3	0	1	1	0
main	minPrice #13	5	4	1	1	1	0
main	maxPrice #14	5	4	1	1	1	0
main	file #15	4	3	0	1	1	0
main	searchBody #16	5	4	1	1	1	0
main	file #17	4	3	0	1	1	0
main	searchSegment #18	5	4	1	1	1	0
main	file #19	4	3	0	1	1	0

$\sum_x span(x) = 149; Q = 104$. Номер после имени различает объявления в разных областях видимости.

Модульная

Функция	Переменная	k	k – 1	P	M	C	T
main	choice #1	6	5	0	1	1	0
addCar	car #1	19	18	1	1	1	0
addCar	file #2	4	3	0	1	1	0
showAll	car #1	3	2	1	1	1	0
showAll	file #2	4	3	0	1	1	0
findCar	searchChoice #1	5	4	0	1	1	0
searchByBrand	car #1	4	3	1	1	1	0
searchByBrand	searchBrand #2	5	4	1	1	1	0
searchByBrand	file #3	4	3	0	1	1	0
searchByModel	car #1	4	3	1	1	1	0
searchByModel	searchModel #2	5	4	1	1	1	0
searchByModel	file #3	4	3	0	1	1	0
searchByYear	car #1	5	4	1	1	1	0
searchByYear	minYear #2	7	6	1	1	1	0
searchByYear	maxYear #3	6	5	1	1	1	0
searchByYear	file #4	4	3	0	1	1	0
searchByPrice	car #1	5	4	1	1	1	0
searchByPrice	minPrice #2	5	4	1	1	1	0
searchByPrice	maxPrice #3	5	4	1	1	1	0
searchByPrice	file #4	4	3	0	1	1	0
searchByBody	car #1	4	3	1	1	1	0
searchByBody	searchBody #2	5	4	1	1	1	0
searchByBody	file #3	4	3	0	1	1	0
searchBySegment	car #1	4	3	1	1	1	0
searchBySegment	searchSegment #2	5	4	1	1	1	0
searchBySegment	file #3	4	3	0	1	1	0
printCar	car #0	6	5	1	0	0	0

$\sum_x span(x) = 113; Q = 147$. Номер после имени различает объявления в разных областях видимости.

Приложение В Подграфы для метрики граничных значений

В таблицах приведены вершины отбора, непосредственные переходы, нижние границы и сложности подграфов. Нумерация соответствует схемам приложения Г.

Линейная main

Отбор	Переходы	Нижняя граница	Сложность
3	2 217	2	141
4	58 66 207 208 209	3	137
5	6 9	3	3
11	10 13	10	5
13	16 17	12	3
20	19 22	19	5
22	25 26	21	3
30	29 33	29	2
35	34 38	34	2
40	39 42	39	5
42	45 46	41	3
49	48 51	48	5
51	54 55	50	3
59	60 65	3	6
62	61 63	61	2
67	3 87	3	15
68	69 75	3	7
71	70 72	70	3
72	71 73	71	2
77	76 79	76	5
79	82 83	78	3
88	67 108	67	15
89	90 96	67	7
92	91 93	91	3
93	92 94	92	2
98	97 100	97	5
100	103 104	99	3
109	88 130	88	15
110	111 117	88	7
113	112 114	112	3
114	113 115	113	2
119	118 122	118	2
124	123 127	123	2
131	109 152	109	15
132	133 139	109	7
135	134 136	134	3
136	135 137	135	2
141	140 144	140	2
146	145 149	145	2
153	131 173	131	15

Отбор	Переходы	Нижняя граница	Сложность
154	155 161	131	7
157	156 158	156	3
158	157 159	157	2
163	162 165	162	5
165	168 169	164	3
174	153 194	153	15
175	176 182	153	7
178	177 179	177	3
179	178 180	178	2
184	183 186	183	5
186	189 190	185	3
195	174 198	174	2
210	4 213	4	2

$$S_a = 531 + 91 = 622; S_0 = 0,768489.$$

С указателями main

Отбор	Переходы	Нижняя граница	Сложность
4	3 218	3	141
5	59 67 208 209 210	4	137
6	7 10	4	3
12	11 14	11	5
14	17 18	13	3
21	20 23	20	5
23	26 27	22	3
31	30 34	30	2
36	35 39	35	2
41	40 43	40	5
43	46 47	42	3
50	49 52	49	5
52	55 56	51	3
60	61 66	4	6
63	62 64	62	2
68	4 88	4	15
69	70 76	4	7
72	71 73	71	3
73	72 74	72	2
78	77 80	77	5
80	83 84	79	3
89	68 109	68	15
90	91 97	68	7
93	92 94	92	3
94	93 95	93	2
99	98 101	98	5
101	104 105	100	3
110	89 131	89	15

Отбор	Переходы	Нижняя граница	Сложность
111	112 118	89	7
114	113 115	113	3
115	114 116	114	2
120	119 123	119	2
125	124 128	124	2
132	110 153	110	15
133	134 140	110	7
136	135 137	135	3
137	136 138	136	2
142	141 145	141	2
147	146 150	146	2
154	132 174	132	15
155	156 162	132	7
158	157 159	157	3
159	158 160	158	2
164	163 166	163	5
166	169 170	165	3
175	154 195	154	15
176	177 183	154	7
179	178 180	178	3
180	179 181	179	2
185	184 187	184	5
187	190 191	186	3
196	175 199	175	2
211	5 214	5	2

$$S_a = 531 + 91 = 622; S_0 = 0,768489.$$

Модульная main

Отбор	Переходы	Нижняя граница	Сложность
3	2 17	2	10
4	5 6 7 8 9	3	6
10	4 13	4	2

$$S_a = 18 + 10 = 28; S_0 = 0,535714.$$

Модульная addCar

Отбор	Переходы	Нижняя граница	Сложность
2	3 6	1	3
8	7 10	7	5
10	13 14	9	3
17	16 19	16	5
19	22 23	18	3
27	26 30	26	2
32	31 35	31	2

Отбор	Переходы	Нижняя граница	Сложность
37	36 39	36	5
39	42 43	38	3
46	45 48	45	5
48	51 52	47	3

$$S_a = 39 + 24 = 63; S_0 = 0,444444.$$

Модульная showAll

Отбор	Переходы	Нижняя граница	Сложность
2	3 7	1	6
5	4 6	4	2

$$S_a = 8 + 6 = 14; S_0 = 0,428571.$$

Модульная findCar

Отбор	Переходы	Нижняя граница	Сложность
2	1 3 4 5 6 7 8	1	7
9	2 12	2	2

$$S_a = 9 + 9 = 18; S_0 = 0,388889.$$

Модульная searchByBrand

Отбор	Переходы	Нижняя граница	Сложность
2	3 8	1	7
5	4 6	4	3
6	5 7	5	2
10	9 12	9	5
12	15 16	11	3

$$S_a = 20 + 10 = 30; S_0 = 0,500000.$$

Модульная searchByModel

Отбор	Переходы	Нижняя граница	Сложность
2	3 8	1	7
5	4 6	4	3
6	5 7	5	2
10	9 12	9	5
12	15 16	11	3

$$S_a = 20 + 10 = 30; S_0 = 0,500000.$$

Модульная searchByYear

Отбор	Переходы	Нижняя граница	Сложность
2	3 8	1	7

Отбор	Переходы	Нижняя граница	Сложность
5	4 6	4	3
6	5 7	5	2
10	9 13	9	2
15	14 18	14	2

$$S_a = 16 + 10 = 26; S_0 = 0,423077.$$

Модульная searchByPrice

Отбор	Переходы	Нижняя граница	Сложность
2	3 8	1	7
5	4 6	4	3
6	5 7	5	2
10	9 13	9	2
15	14 18	14	2

$$S_a = 16 + 10 = 26; S_0 = 0,423077.$$

Модульная searchByBody

Отбор	Переходы	Нижняя граница	Сложность
2	3 8	1	7
5	4 6	4	3
6	5 7	5	2
10	9 12	9	5
12	15 16	11	3

$$S_a = 20 + 10 = 30; S_0 = 0,500000.$$

Модульная searchBySegment

Отбор	Переходы	Нижняя граница	Сложность
2	3 8	1	7
5	4 6	4	3
6	5 7	5	2
10	9 12	9	5
12	15 16	11	3

$$S_a = 20 + 10 = 30; S_0 = 0,500000.$$

Модульная printHeader

$$S_a = 0 + 2 = 2; S_0 = 0.$$

Модульная printCar

$$S_a = 0 + 2 = 2; S_0 = 0.$$

Приложение Г Блок схемы и графы управления

Ромб — условие или выбор; прямоугольник — действия; овал — начало или конец. Пунктирные кружки — соединители фрагментов, не дополнительные вершины графа. Листинги приведены в приложении Д.

На общих схемах линейной и указательной версий номера записаны в формате Л/У. Создание и удаление динамического объекта отмечены в блоках. Модульные функции показаны отдельно.

Рисунок 1 Линейная и указательная версии Главное меню

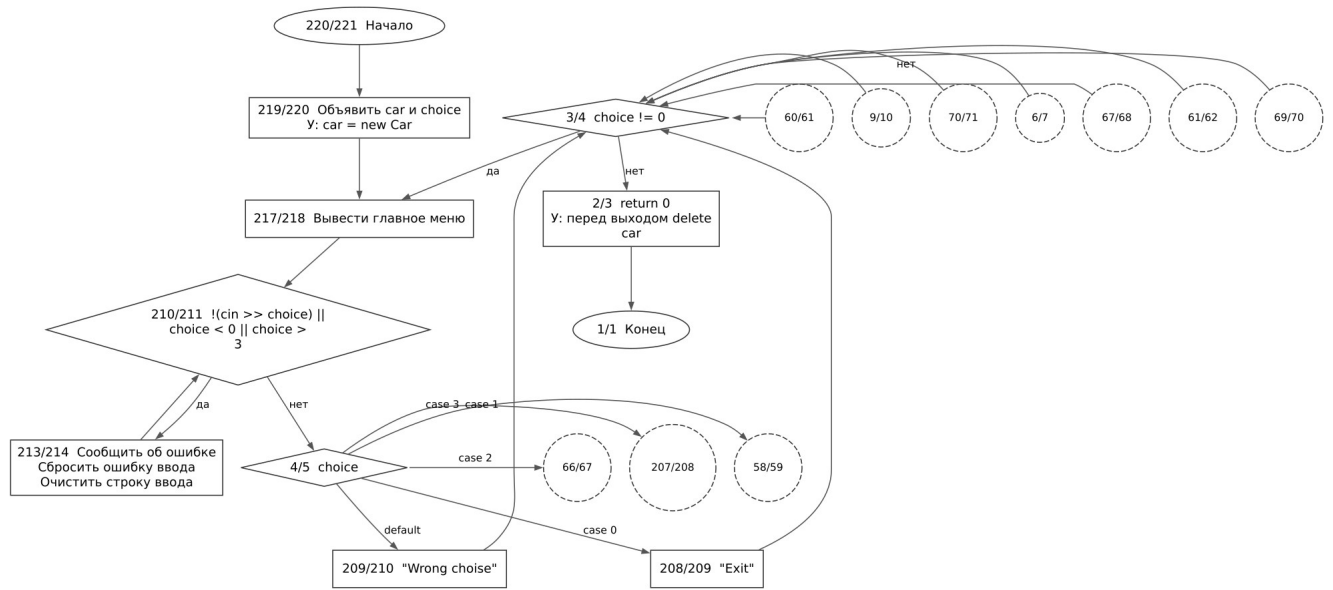


Рисунок 2 Линейная и указательная версии Выбор атрибута поиска

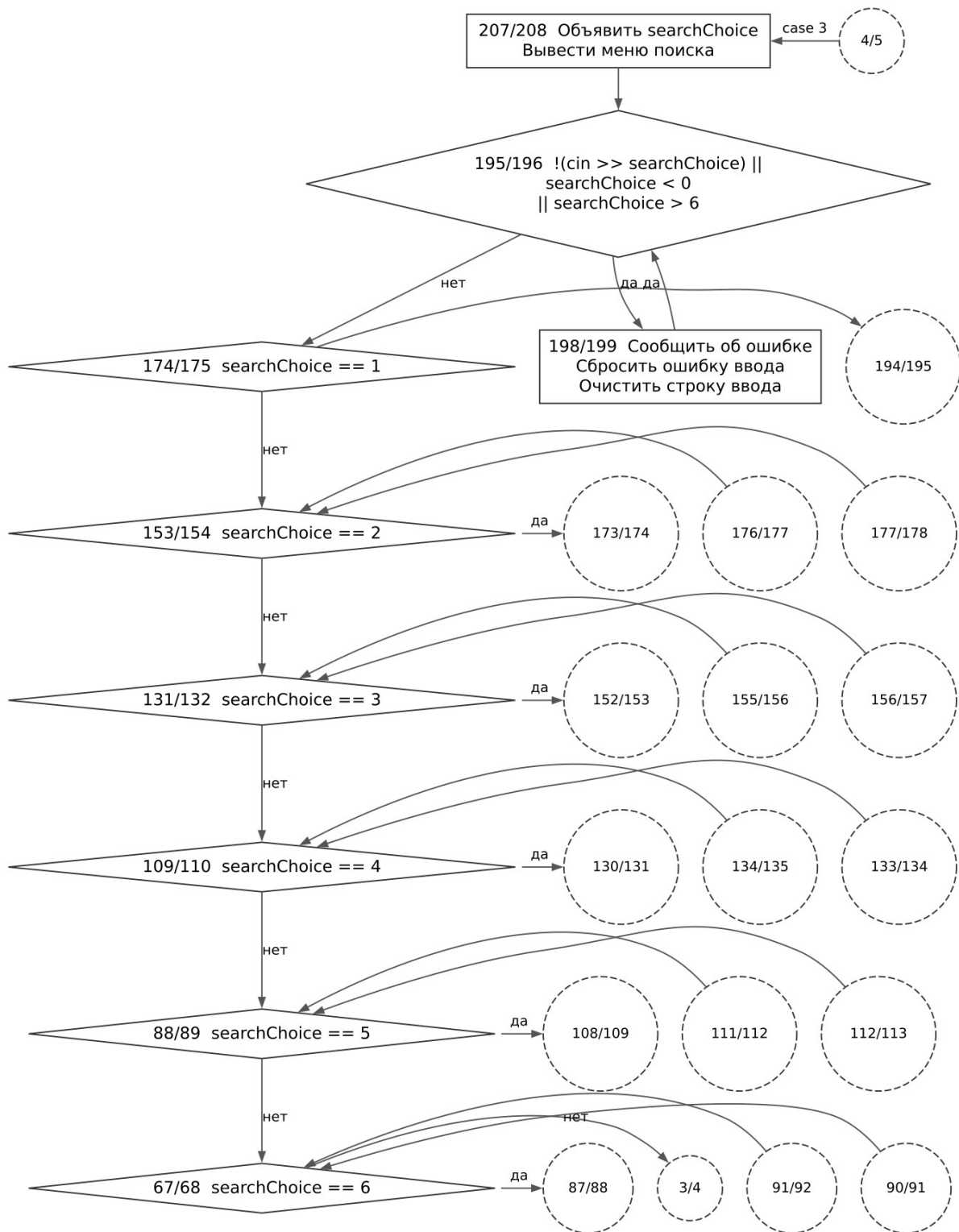


Рисунок 3 Линейная и указательная версии Добавление марки модели и года

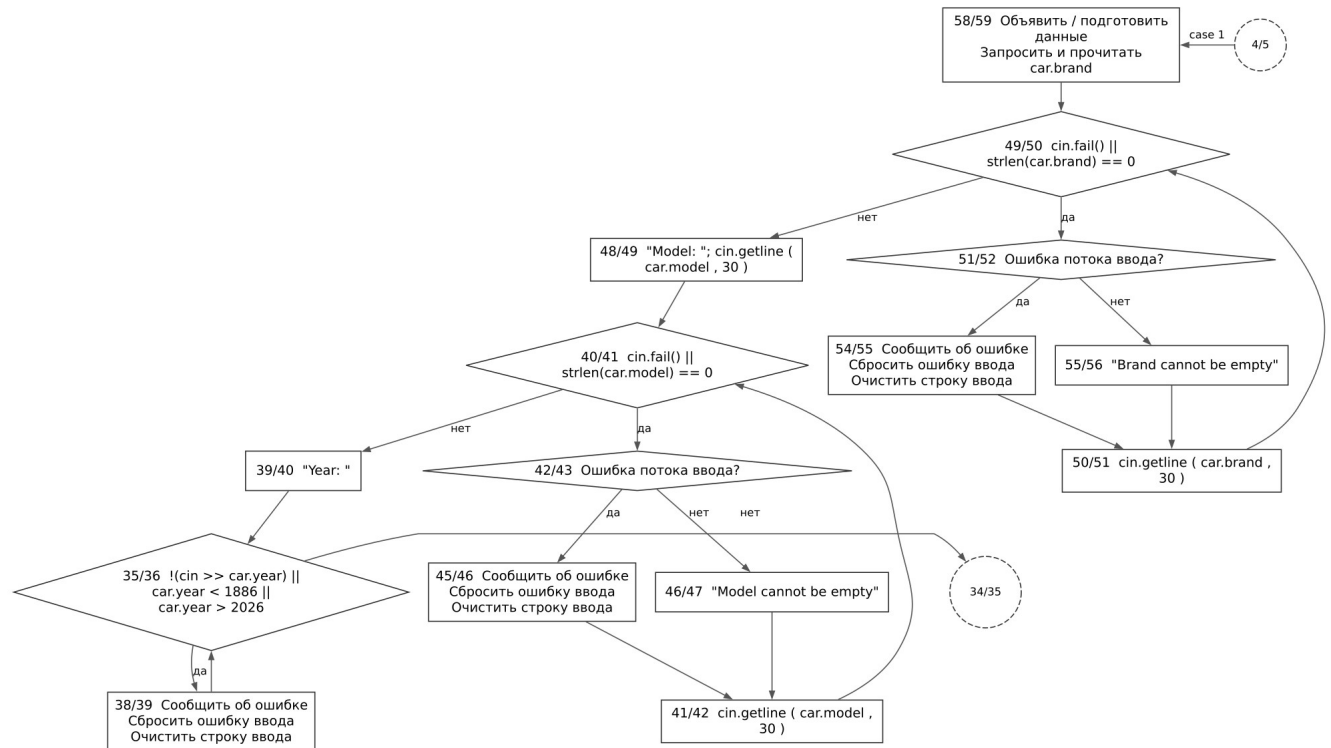


Рисунок 4 Линейная и указательная версии Добавление цены кузова сегмента и запись

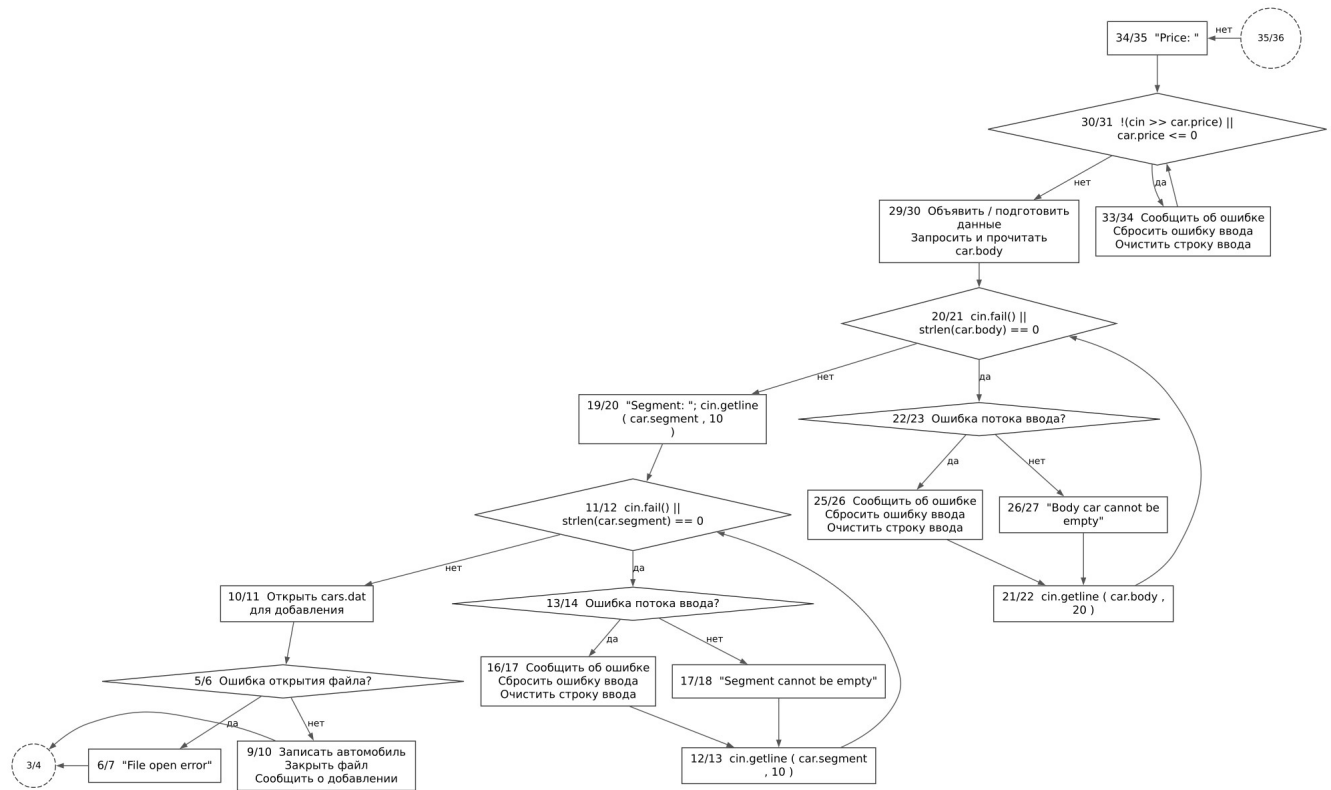


Рисунок 5 Линейная и указательная версии Просмотр всех записей

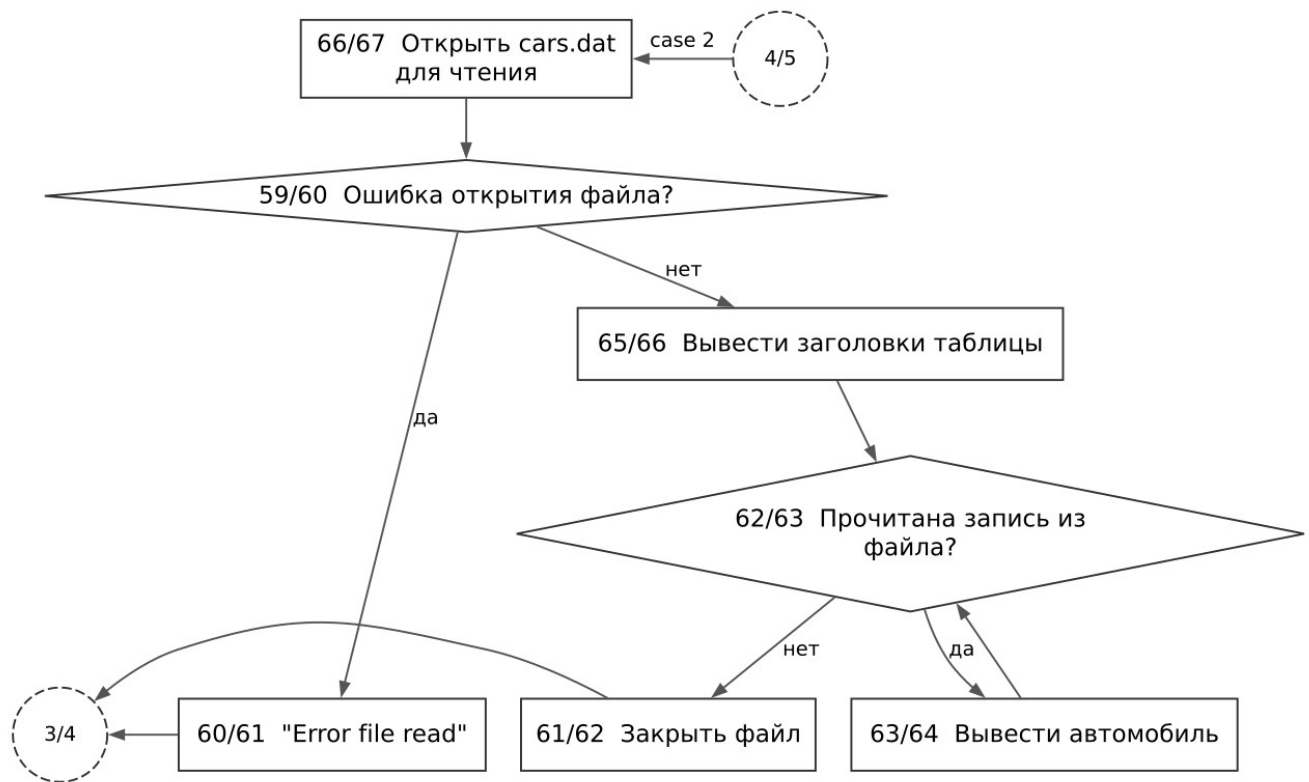


Рисунок 6 Линейная и указательная версии Поиск марки

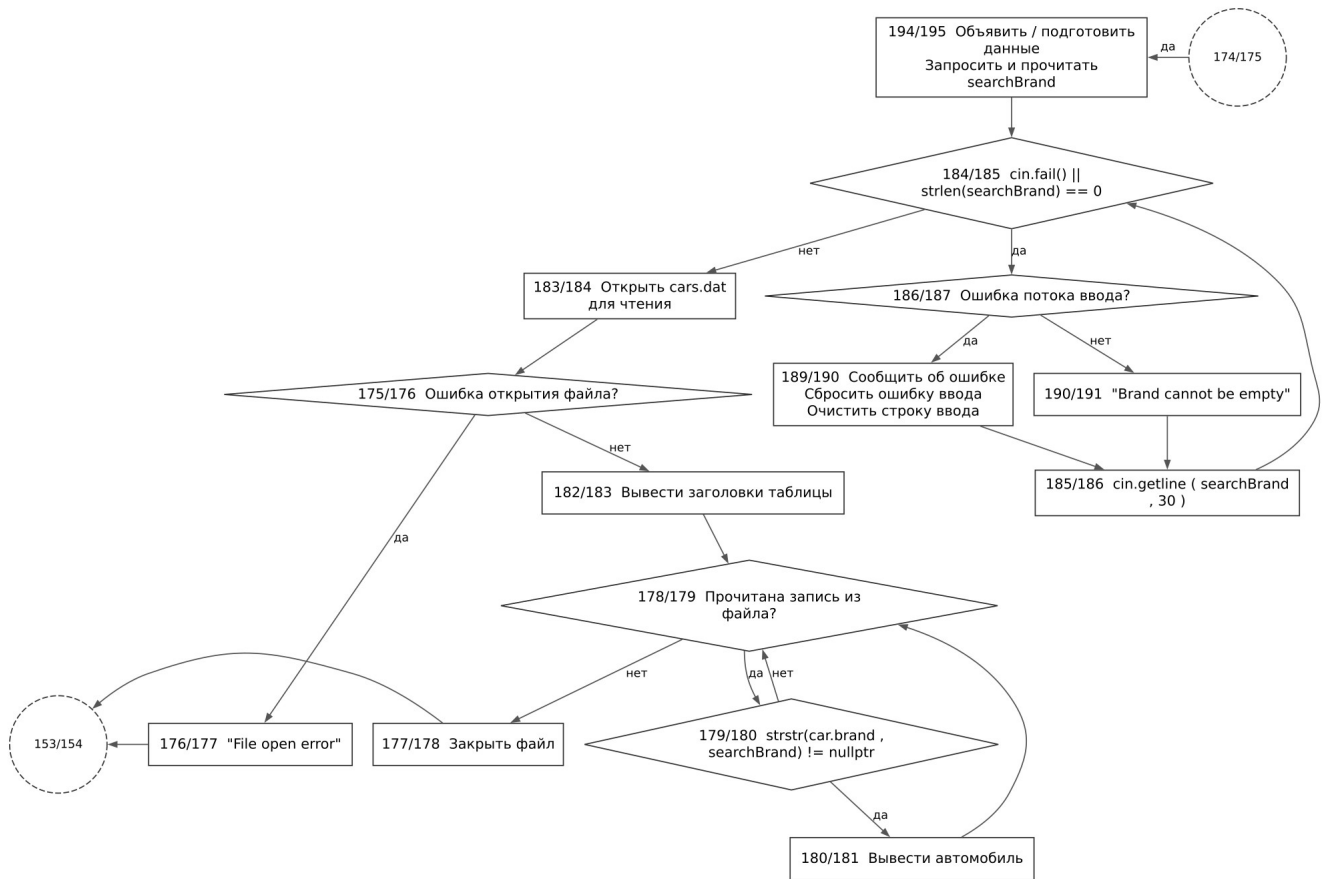


Рисунок 7 Линейная и указательная версии Поиск модели

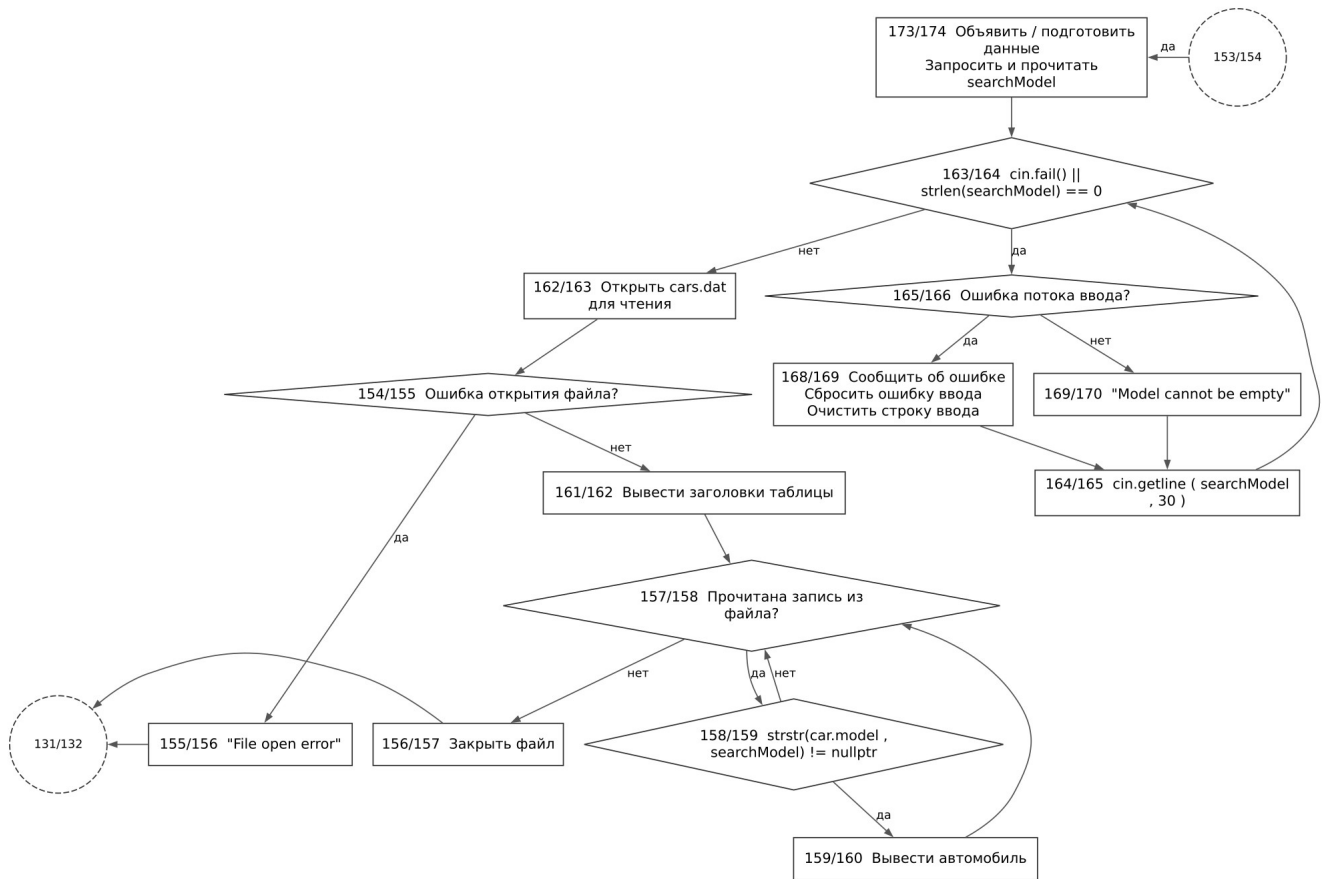


Рисунок 8 Линейная и указательная версии Поиск года

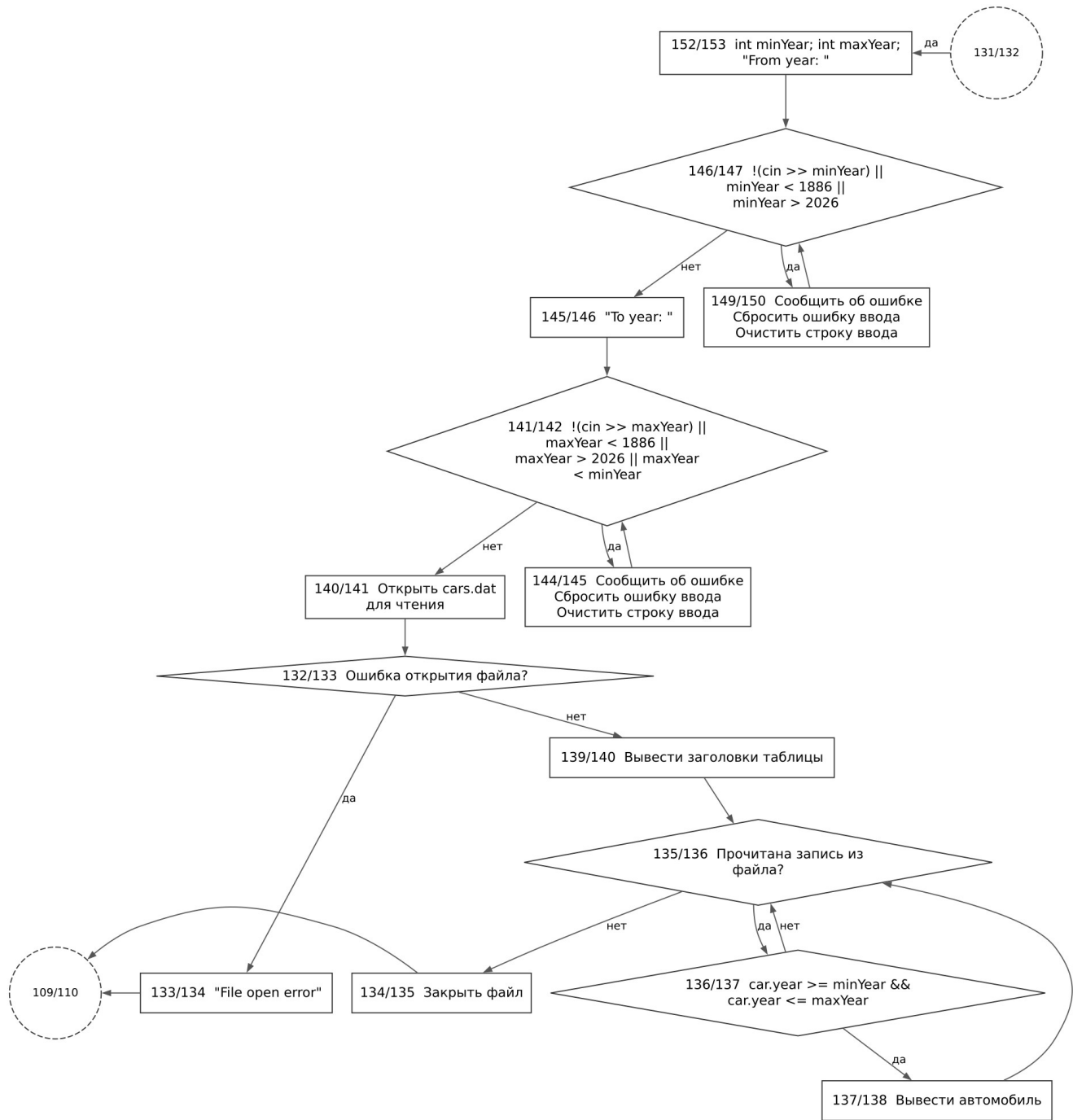


Рисунок 9 Линейная и указательная версии Поиск цены

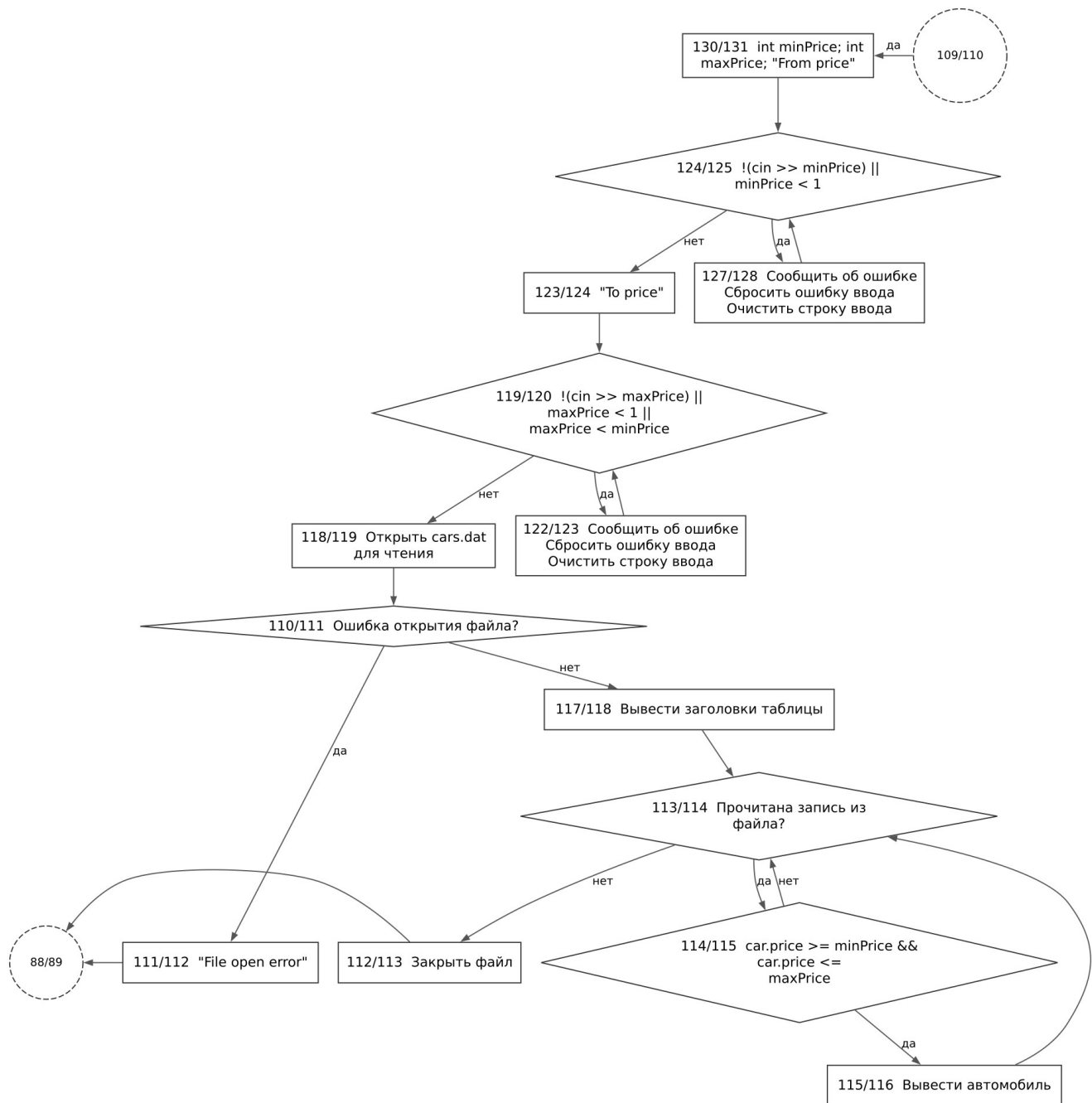


Рисунок 10 Линейная и указательная версии Поиск кузова

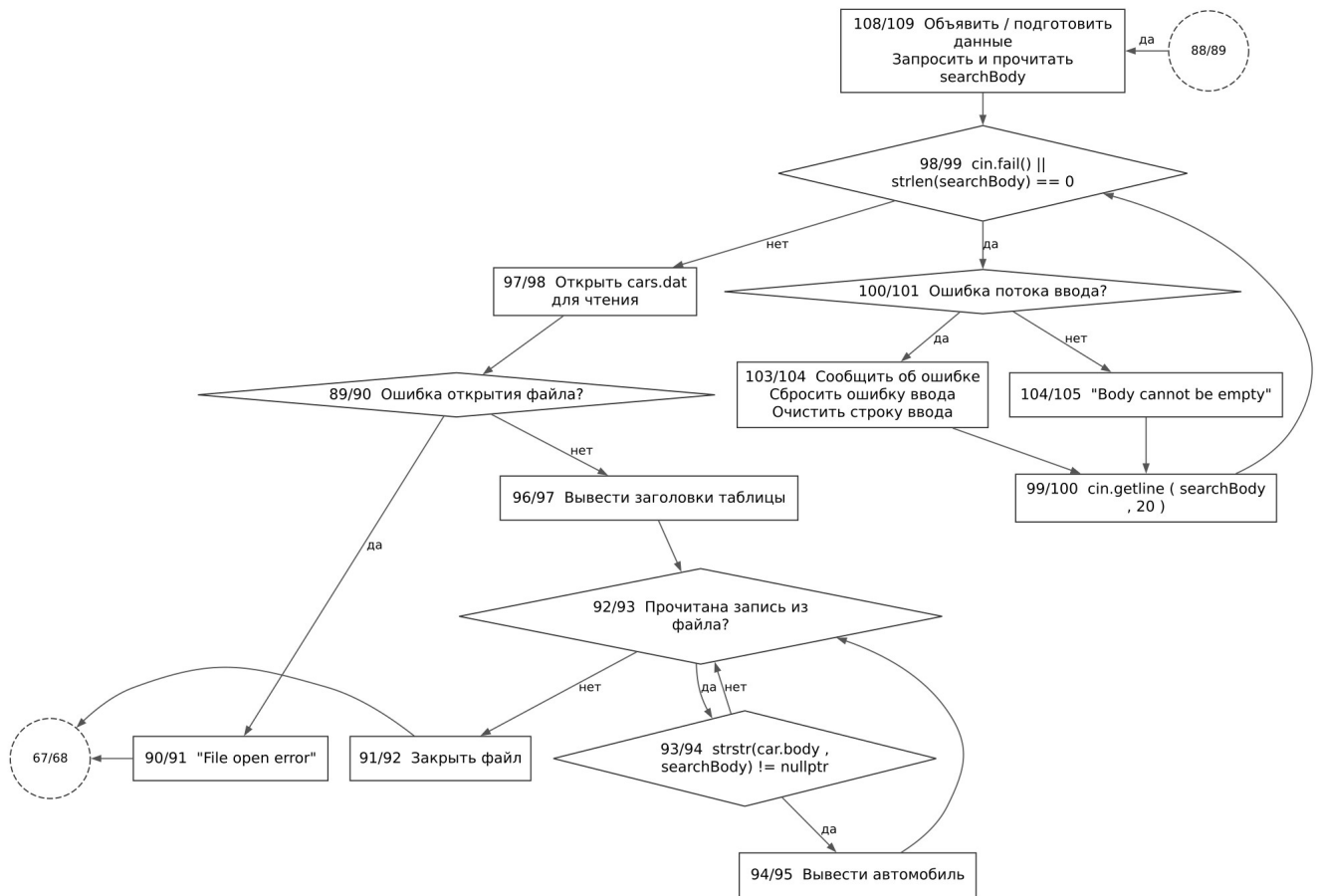


Рисунок 11 Линейная и указательная версии Поиск сегмента

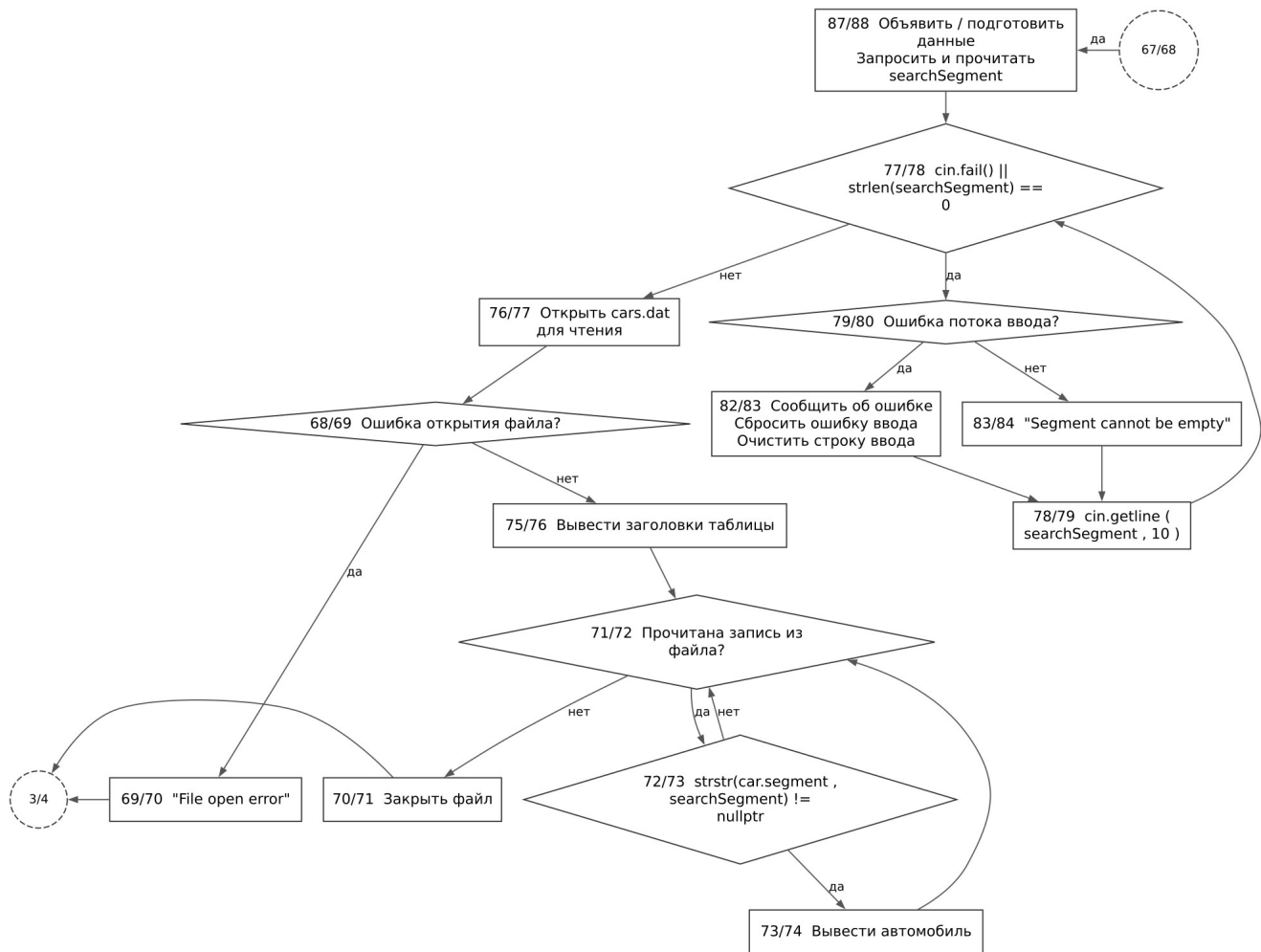


Рисунок 12 Модульная версия main

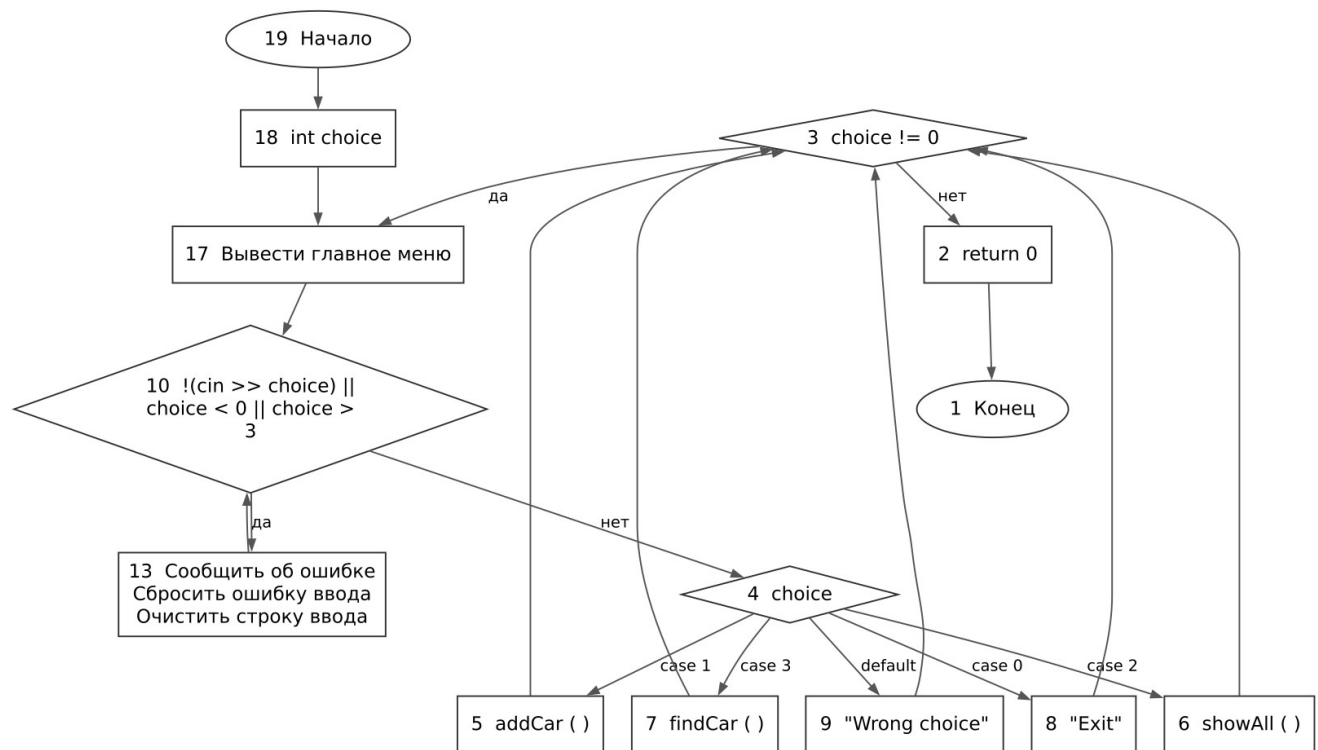


Рисунок 13 Модульная версия addCar ввод марки модели и года

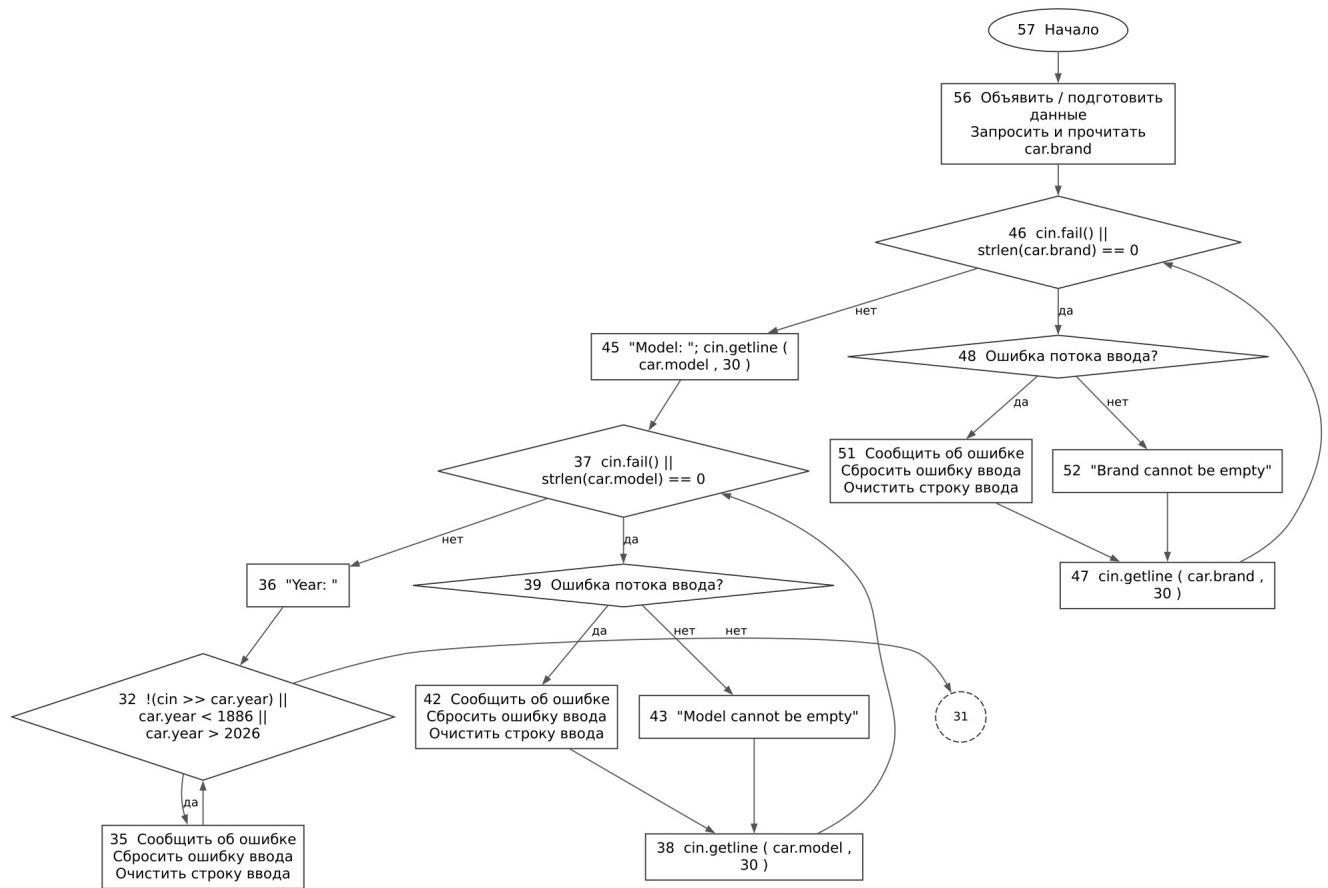


Рисунок 14 Модульная версия addCar ввод цены кузова сегмента и запись

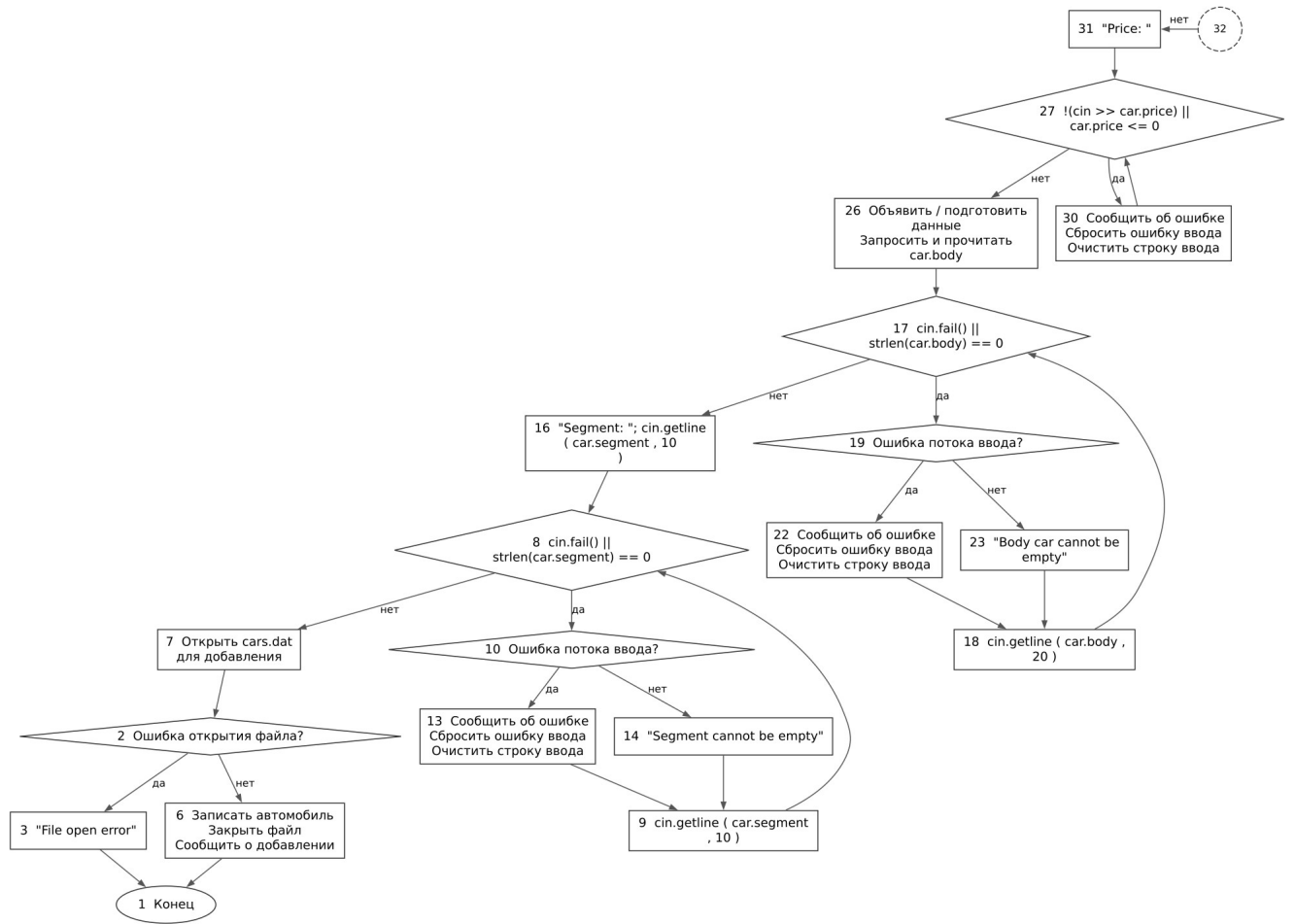


Рисунок 15 Модульная версия showAll

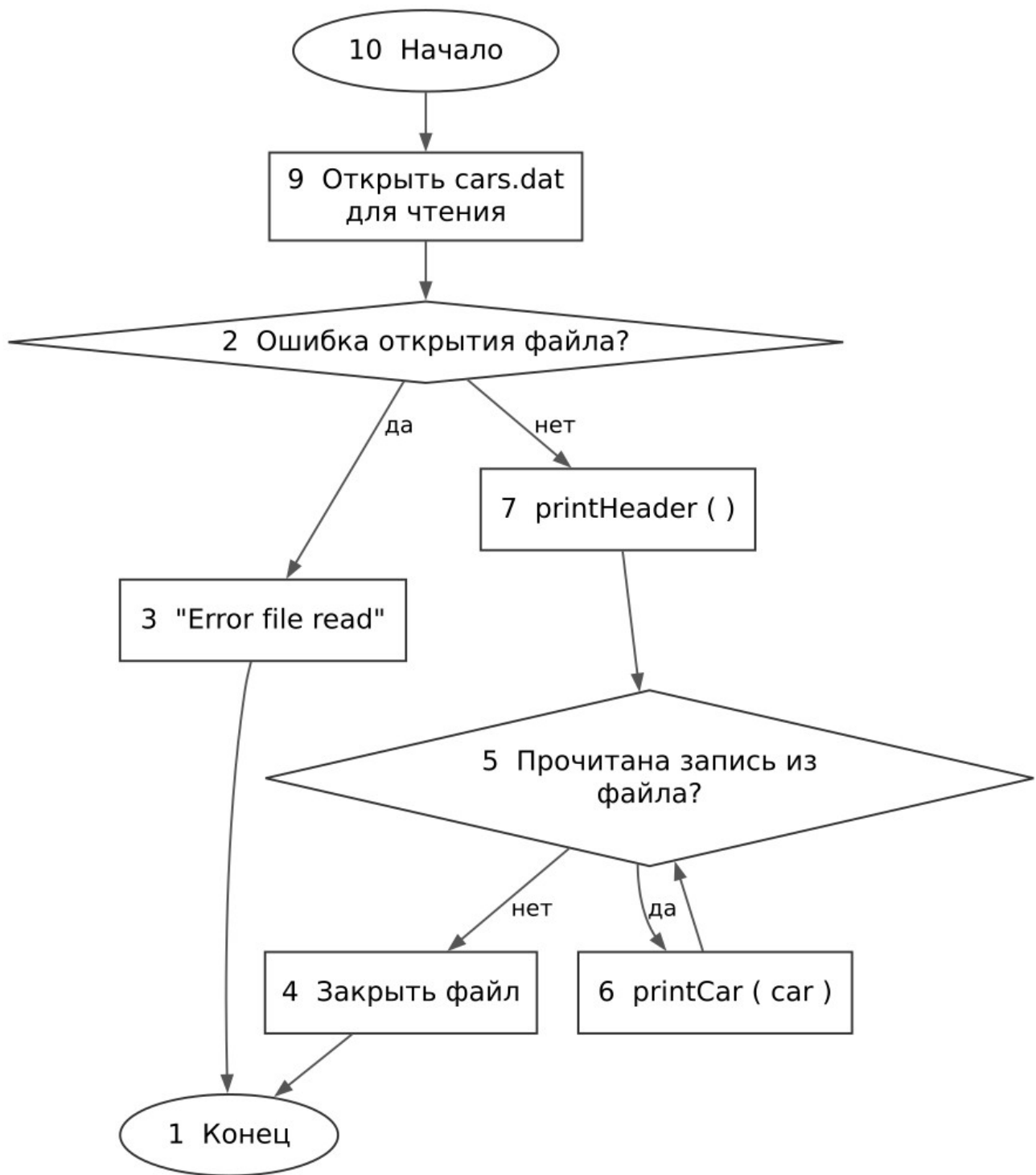


Рисунок 16 Модульная версия findCar

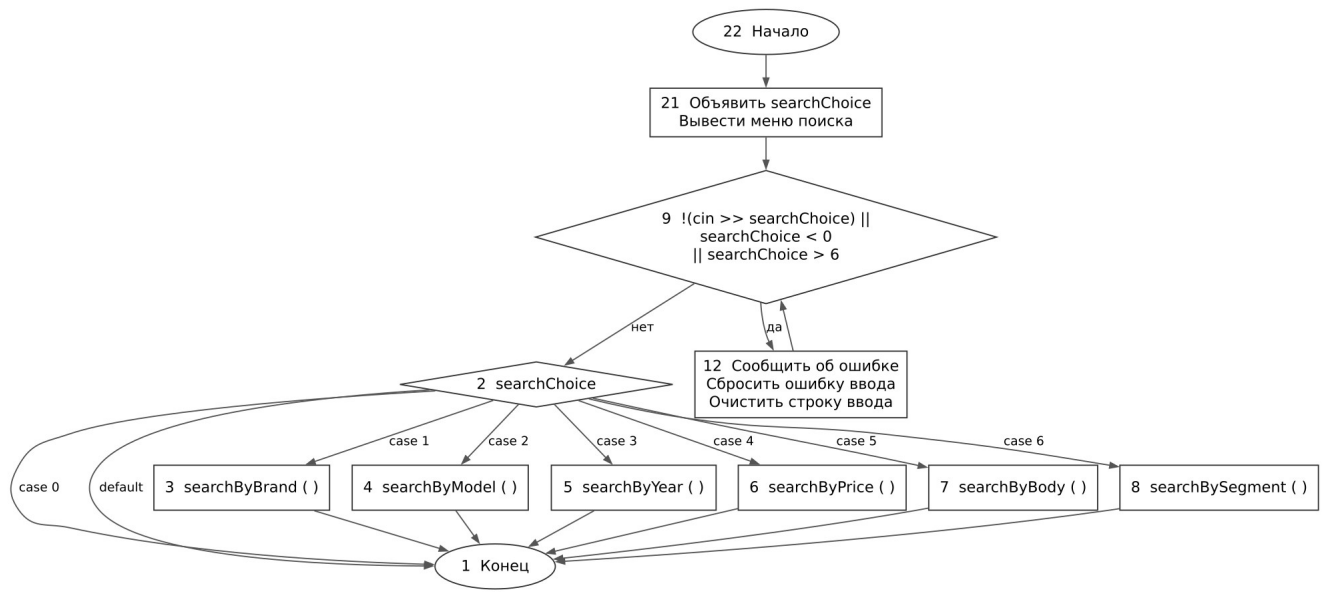


Рисунок 17 Модульная версия searchByBrand

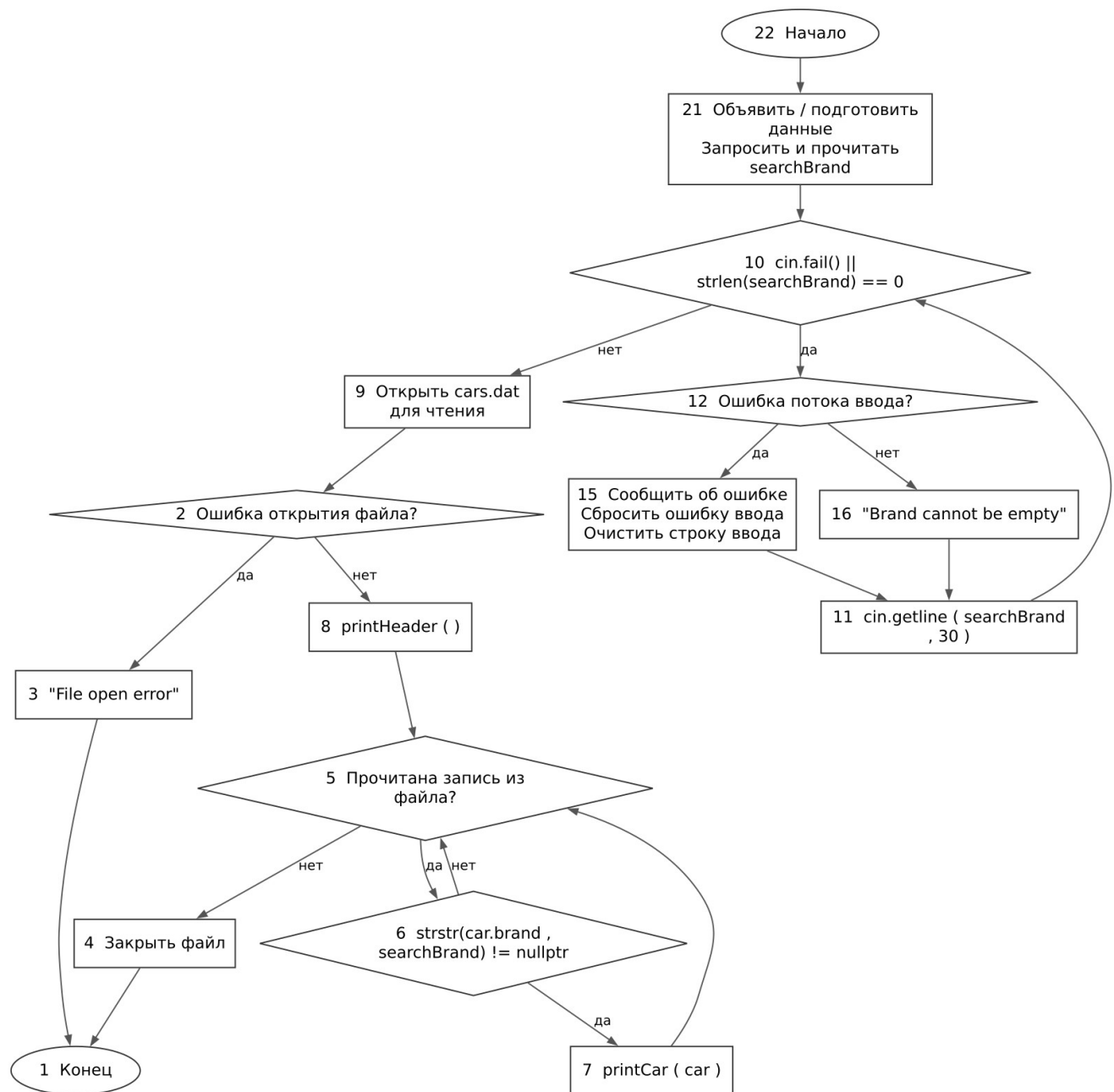


Рисунок 18 Модульная версия searchByModel

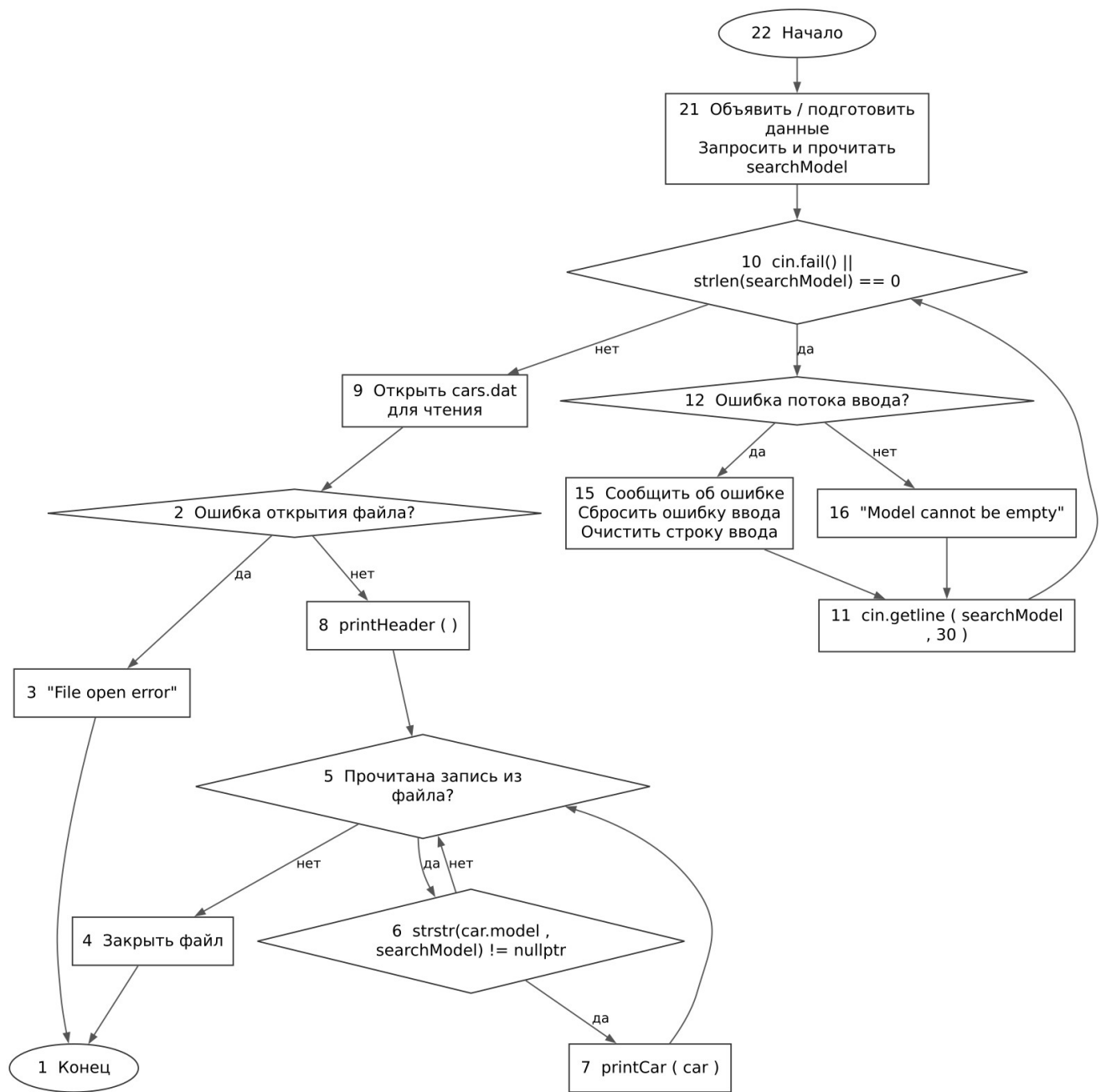


Рисунок 19 Модульная версия searchByYear

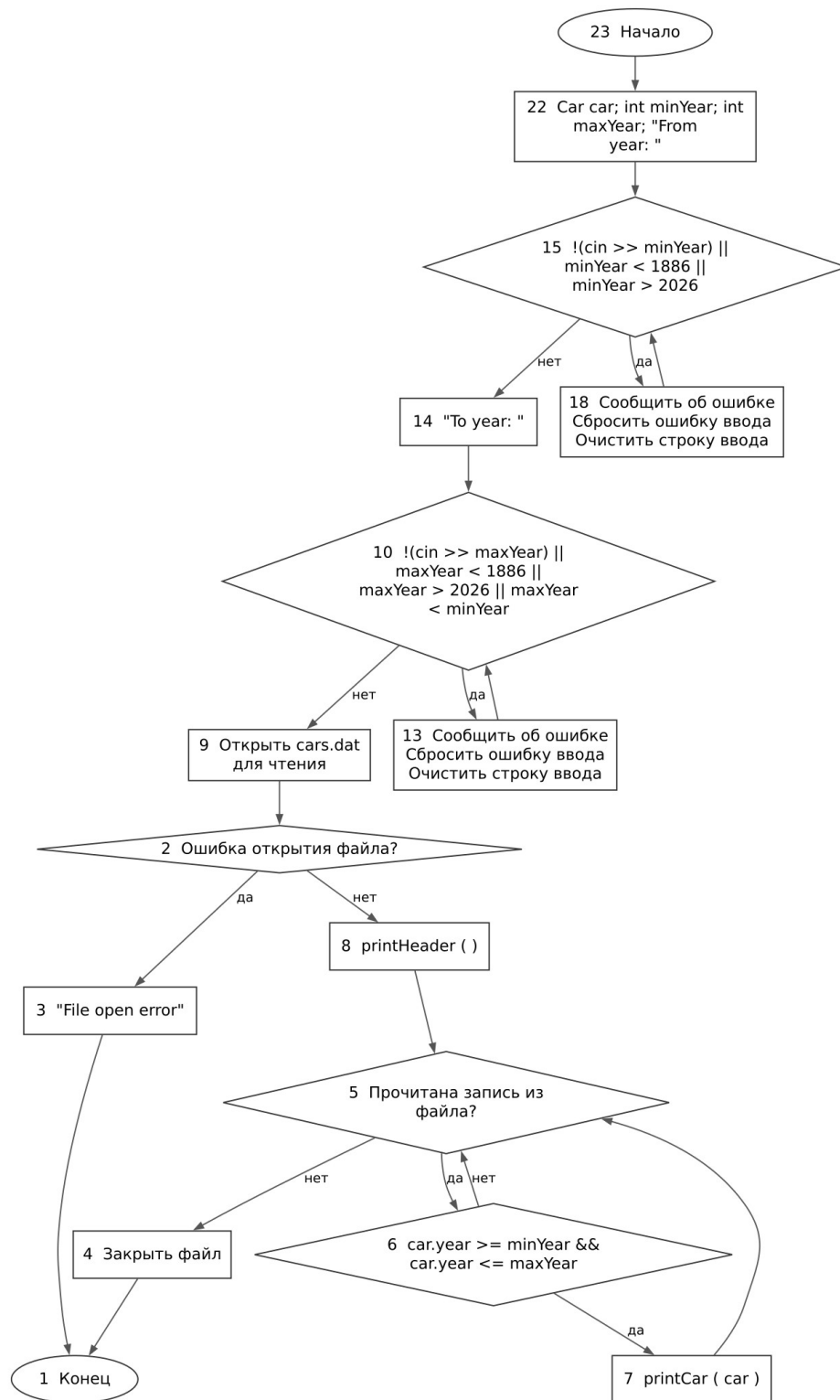


Рисунок 20 Модульная версия searchByPrice

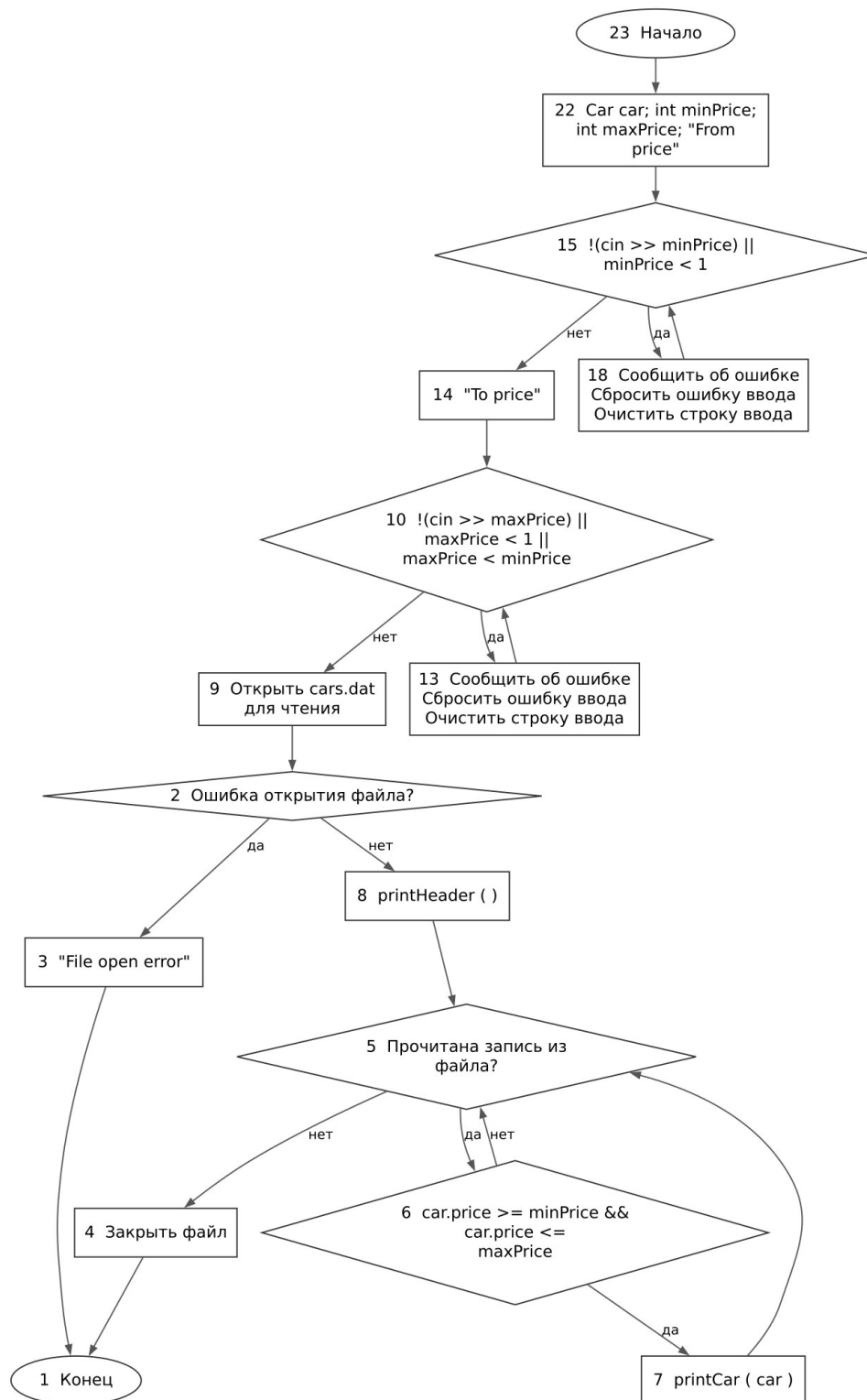


Рисунок 21 Модульная версия searchByBody

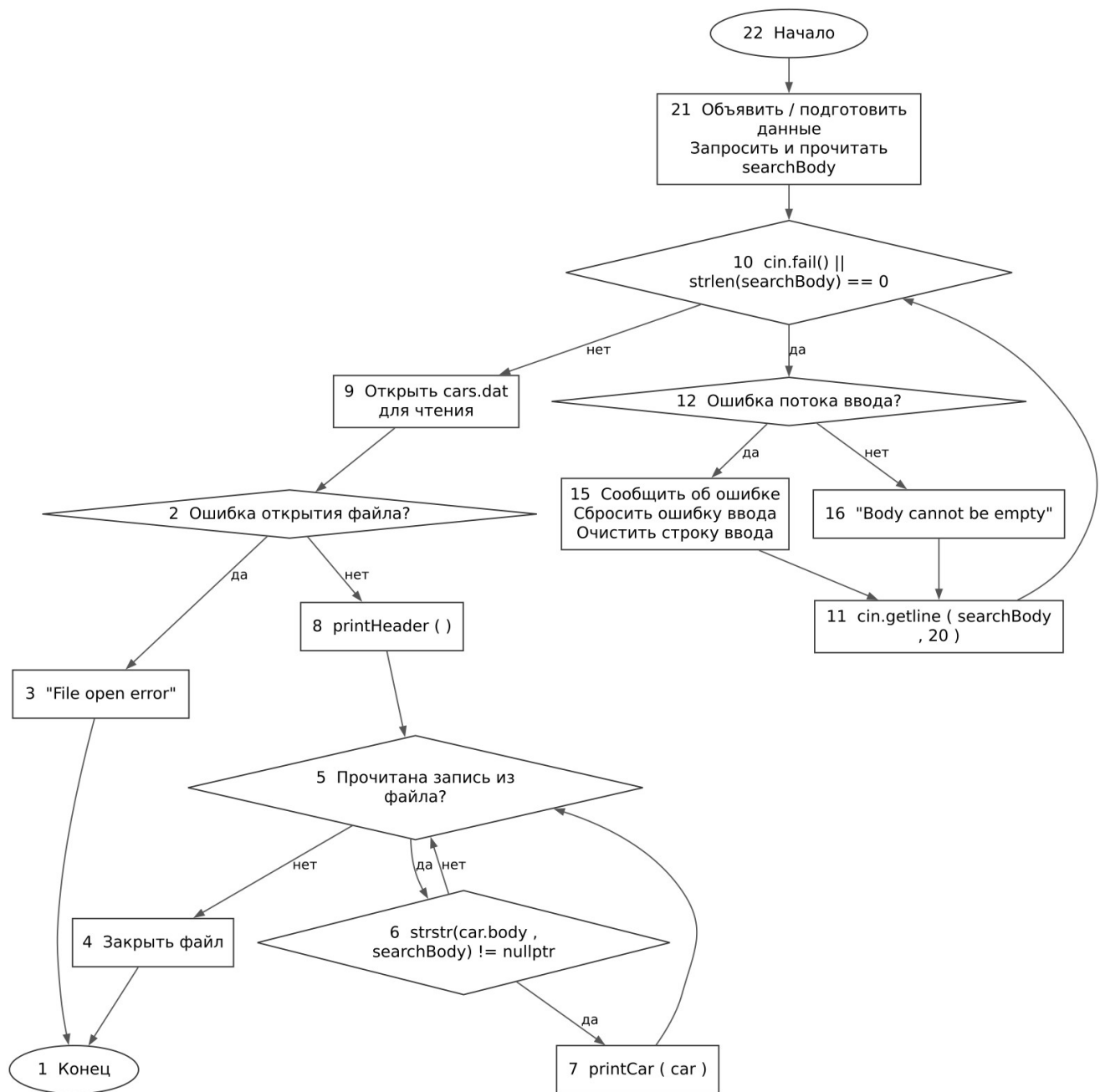


Рисунок 22 Модульная версия searchBySegment

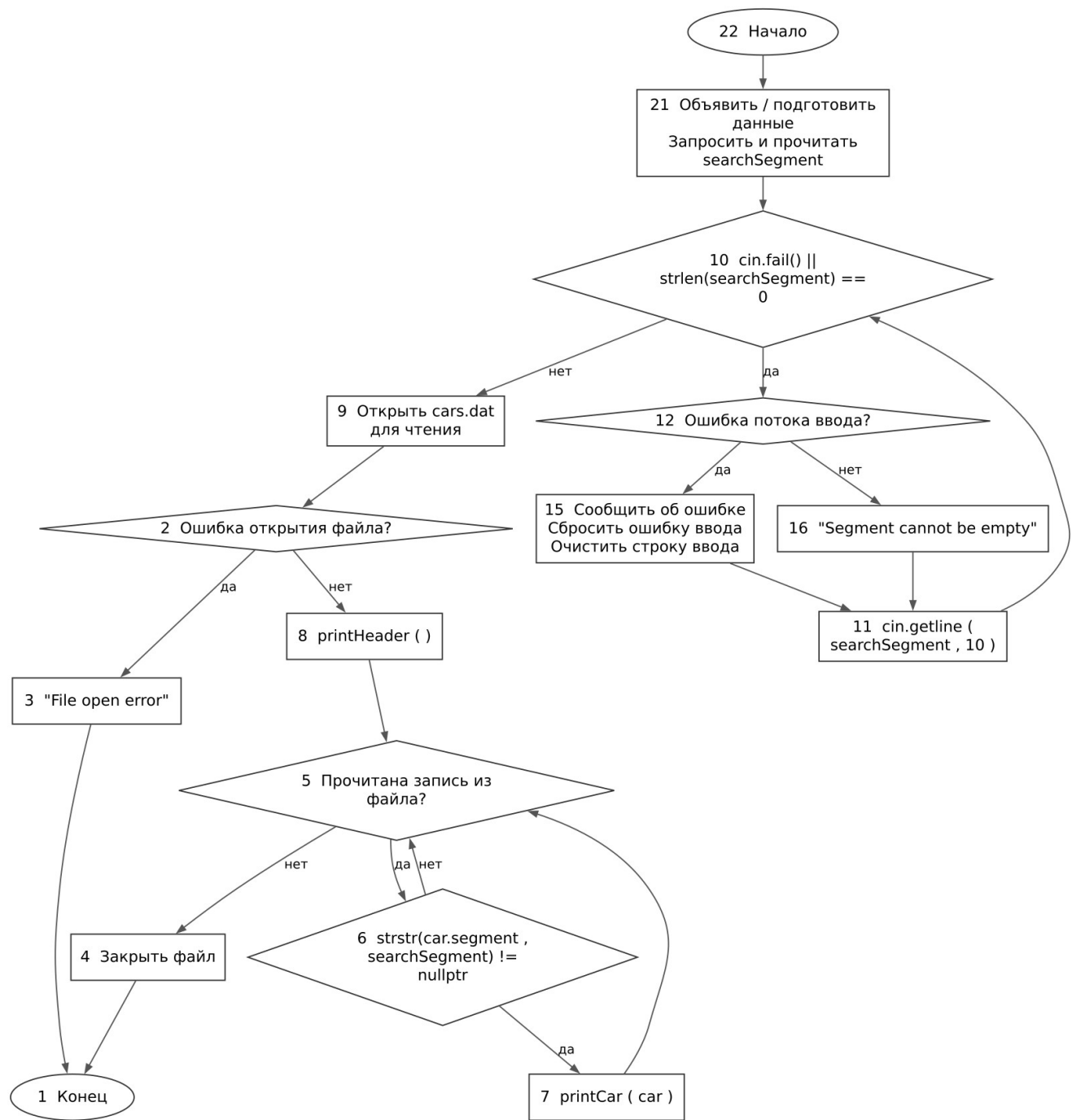


Рисунок 23 Модульная версия printHeader



Рисунок 24 Модульная версия printCar



Приложение Д Листинги программ

lab1/linear.cpp

```
#include <iostream>
#include <fstream>
#include <cstring>
#include <limits>
#include <iomanip>
#include <string>

using namespace std;

struct Car {
    char brand[30];
    char model[30];
    int year;
    int price;
    char body[20];
    char segment[10];
};

int main() {
    Car car;
    int choice;

    do {
        cout << "1 - Add car" << endl;
        cout << "2 - Show all" << endl;
        cout << "3 - Search" << endl;
        cout << "0 - Exit" << endl;

        while (!(cin >> choice) || choice < 0 || choice > 3) {
            cout << "Enter a number: ";

            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
        switch (choice) {
            case 1: {
                cin.ignore(numeric_limits<streamsize>::max(), '\n');

                cout << "Brand: ";
                cin.getline(car.brand, 30);

                while (cin.fail() || strlen(car.brand) == 0) {
                    if (cin.fail()) {
                        cin.clear();
                        cin.ignore(numeric_limits<streamsize>::max(), '\n');

                        cout << "Brand is too long. Maximum 29 characters: ";
                    }
                    else {
                        cout << "Brand cannot be empty" << endl;
                    }

                    cin.getline(car.brand, 30);
                }

                cout << "Model: ";
                cin.getline(car.model, 30);

                while (cin.fail() || strlen(car.model) == 0) {
                    if (cin.fail()) {
                        cin.clear();
                        cin.ignore(numeric_limits<streamsize>::max(), '\n');

                        cout << "Model is too long. Maximum 29 characters" << endl;
                    }
                    else {
                        cout << "Model cannot be empty" << endl;
                    }
                }
            }
        }
    }
}
```

```

        cin.getline(car.model, 30);
    }

    cout << "Year: ";
    while (!(cin >> car.year) || car.year < 1886 || car.year > 2026) {
        cout << "Wrong year. Enter year from 1886 to 2026" << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << "Price: ";
    while (!(cin >> car.price) || car.price <= 0) {
        cout << "Wrong price. Enter positive number: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Body: ";
    cin.getline(car.body, 20);

    while (cin.fail() || strlen(car.body) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Body car is too long. Maximum 19 characters" << endl;
        }
        else {
            cout << "Body car cannot be empty" << endl;
        }
        cin.getline(car.body, 20);
    }

    cout << "Segment: ";
    cin.getline(car.segment, 10);

    while (cin.fail() || strlen(car.segment) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Segment is too long. Maximum 9 characters" << endl;
        }
        else {
            cout << "Segment cannot be empty" << endl;
        }
        cin.getline(car.segment, 10);
    }

    ofstream file("cars.dat", ios::binary | ios::app);
    if (!file) {
        cout << "File open error" << endl;
    }
    else {
        file.write(reinterpret_cast<char*>(&car), sizeof(Car));

        file.close();

        cout << "Car added." << endl;
    }

    break;
}
case 2: {
    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "Error file read" << endl;
    }
}

```

```

else {
    cout << left
        << setw(31) << "Brand"
        << setw(31) << "Model"
        << setw(21) << "Body"
        << setw(11) << "Segment"
        << setw(8) << "Year"
        << setw(14) << "Price"
        << endl;
    cout << string(116, '-') << endl;
    while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
        cout << left
            << setw(31) << car.brand
            << setw(31) << car.model
            << setw(21) << car.body
            << setw(11) << car.segment
            << setw(8) << car.year
            << setw(14) << car.price
            << endl;
    }

    file.close();
}
break;
}
case 3: {
    int searchChoice;

    cout << "search by: " << endl;
    cout << "1 - brand" << endl;
    cout << "2 - model" << endl;
    cout << "3 - year" << endl;
    cout << "4 - price" << endl;
    cout << "5 - body" << endl;
    cout << "6 - segment " << endl;
    cout << "0 - back" << endl;

    while (!(cin >> searchChoice) || searchChoice < 0 || searchChoice > 6) {
        cout << "Wrong choice. Enter 0-6: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    if (searchChoice == 1) {
        char searchBrand[30];

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Enter Brand: ";
        cin.getline(searchBrand, 30);

        while (cin.fail() || strlen(searchBrand) == 0) {
            if (cin.fail()) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');

                cout << "Brand is too long. Maximum 29 characters" << endl;
            }
            else {
                cout << "Brand cannot be empty" << endl;
            }
            cin.getline(searchBrand, 30);
        }

        ifstream file("cars.dat", ios::binary);

        if (!file) {
            cout << "File open error" << endl;
        }
        else {
            cout << left
                << setw(31) << "Brand"

```

```

        << setw(31) << "Model"
        << setw(21) << "Body"
        << setw(11) << "Segment"
        << setw(8) << "Year"
        << setw(14) << "Price"
        << endl;
    cout << string(116, '-') << endl;
    while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
        if (strstr(car.brand, searchBrand) != nullptr) {

            cout << left
                << setw(31) << car.brand
                << setw(31) << car.model
                << setw(21) << car.body
                << setw(11) << car.segment
                << setw(8) << car.year
                << setw(14) << car.price
                << endl;

        }
    }
    file.close();
}

if (searchChoice == 2) {
    char searchModel[30];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter model: " << endl;

    cin.getline(searchModel, 30);
    while (cin.fail() || strlen(searchModel) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Model is too long. Maximum characters 29" << endl;
        }
        else {
            cout << "Model cannot be empty" << endl;
        }
        cin.getline(searchModel, 30);
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    }
    else {
        cout << left
            << setw(31) << "Brand"
            << setw(31) << "Model"
            << setw(21) << "Body"
            << setw(11) << "Segment"
            << setw(8) << "Year"
            << setw(14) << "Price"
            << endl;
        cout << string(116, '-') << endl;
        while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
            if (strstr(car.model, searchModel) != nullptr) {

                cout << left
                    << setw(31) << car.brand
                    << setw(31) << car.model
                    << setw(21) << car.body
                    << setw(11) << car.segment
                    << setw(8) << car.year
                    << setw(14) << car.price
                    << endl;

            }
        }
        file.close();
    }
}

```

```

}
if (searchChoice == 3) {
    int minYear;
    int maxYear;
    cout << "From year: " << endl;
    while (!(cin >> minYear) || minYear < 1886 || minYear > 2026) {
        cout << "Wrong year. Enter year from 1886 to 2026" << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    cout << "To year: " << endl;
    while (!(cin >> maxYear) || maxYear < 1886 || maxYear > 2026 || maxYear < minYear) {
        cout << "Wrong year. Enter year from " << minYear << " to 2026: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    ifstream file("cars.dat", ios::binary);
    if (!file) {
        cout << "File open error" << endl;
    }
    else {
        cout << left
            << setw(31) << "Brand"
            << setw(31) << "Model"
            << setw(21) << "Body"
            << setw(11) << "Segment"
            << setw(8) << "Year"
            << setw(14) << "Price"
            << endl;
        cout << string(116, '-') << endl;
        while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
            if (car.year >= minYear && car.year <= maxYear) {

                cout << left
                    << setw(31) << car.brand
                    << setw(31) << car.model
                    << setw(21) << car.body
                    << setw(11) << car.segment
                    << setw(8) << car.year
                    << setw(14) << car.price
                    << endl;
            }
        }
        file.close();
    }
}
if (searchChoice == 4) {
    int minPrice;
    int maxPrice;
    cout << "From price" << endl;
    while (!(cin >> minPrice) || minPrice < 1) {
        cout << "Wrong price. Enter correct price." << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    cout << "To price" << endl;
    while (!(cin >> maxPrice) || maxPrice < 1 || maxPrice < minPrice) {
        cout << "Wrong price. Enter correct price." << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    }
    else {
        cout << left

```

```

        << setw(31) << "Brand"
        << setw(31) << "Model"
        << setw(21) << "Body"
        << setw(11) << "Segment"
        << setw(8) << "Year"
        << setw(14) << "Price"
        << endl;
    cout << string(116, '-') << endl;
    while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
        if (car.price >= minPrice && car.price <= maxPrice) {

            cout << left
                << setw(31) << car.brand
                << setw(31) << car.model
                << setw(21) << car.body
                << setw(11) << car.segment
                << setw(8) << car.year
                << setw(14) << car.price
                << endl;

        }
    }
    file.close();
}
}
if (searchChoice == 5) {
    char searchBody[20];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter body: " << endl;
    cin.getline(searchBody, 20);
    while (cin.fail() || strlen(searchBody) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Body is too long. Maximum characters 19" << endl;
        }
        else {
            cout << "Body cannot be empty" << endl;
        }
        cin.getline(searchBody, 20);
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    }
    else {
        cout << left
            << setw(31) << "Brand"
            << setw(31) << "Model"
            << setw(21) << "Body"
            << setw(11) << "Segment"
            << setw(8) << "Year"
            << setw(14) << "Price"
            << endl;
        cout << string(116, '-') << endl;

        while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
            if (strstr(car.body, searchBody) != nullptr) {

                cout << left
                    << setw(31) << car.brand
                    << setw(31) << car.model
                    << setw(21) << car.body
                    << setw(11) << car.segment
                    << setw(8) << car.year
                    << setw(14) << car.price
                    << endl;

            }
        }
        file.close();
    }
}

```

```

    }
}
if (searchChoice == 6) {
    char searchSegment[10];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter segment: " << endl;
    cin.getline(searchSegment, 10);

    while (cin.fail() || strlen(searchSegment) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Segment is too long. Maximum characters 9" << endl;
        }
        else {
            cout << "Segment cannot be empty" << endl;
        }
        cin.getline(searchSegment, 10);
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    }
    else {
        cout << left
            << setw(31) << "Brand"
            << setw(31) << "Model"
            << setw(21) << "Body"
            << setw(11) << "Segment"
            << setw(8) << "Year"
            << setw(14) << "Price"
            << endl;
        cout << string(116, '-') << endl;
        while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
            if (strstr(car.segment, searchSegment) != nullptr) {
                cout << left
                    << setw(31) << car.brand
                    << setw(31) << car.model
                    << setw(21) << car.body
                    << setw(11) << car.segment
                    << setw(8) << car.year
                    << setw(14) << car.price
                    << endl;
            }
        }
        file.close();
    }
}
break;
}
case 0:
{
    cout << "Exit" << endl;
    break;
}

default:
    cout << "Wrong choise" << endl;
    break;
}

} while (choice != 0);

return 0;
}

```


lab1/pointers.cpp

```
#include <iostream>
#include <fstream>
#include <cstring>
#include <limits>
#include <iomanip>
#include <string>

using namespace std;

struct Car {
    char brand[30];
    char model[30];
    int year;
    int price;
    char body[20];
    char segment[10];
};

int main() {
    Car* car = new Car;
    int choice;

    do {
        cout << "1 - Add car" << endl;
        cout << "2 - Show all" << endl;
        cout << "3 - Search" << endl;
        cout << "0 - Exit" << endl;

        while (!(cin >> choice) || choice < 0 || choice > 3) {
            cout << "Enter a number: ";

            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }

        switch (choice) {
            case 1: {
                cin.ignore(numeric_limits<streamsize>::max(), '\n');

                cout << "Brand: ";
                cin.getline(car->brand, 30);

                while (cin.fail() || strlen(car->brand) == 0) {
                    if (cin.fail()) {
                        cin.clear();
                        cin.ignore(numeric_limits<streamsize>::max(), '\n');

                        cout << "Brand is too long. Maximum 29 characters: ";
                    } else {
                        cout << "Brand cannot be empty" << endl;
                    }

                    cin.getline(car->brand, 30);
                }

                cout << "Model: ";
                cin.getline(car->model, 30);

                while (cin.fail() || strlen(car->model) == 0) {
                    if (cin.fail()) {
                        cin.clear();
                        cin.ignore(numeric_limits<streamsize>::max(), '\n');

                        cout << "Model is too long. Maximum 29 characters" << endl;
                    } else {
                        cout << "Model cannot be empty" << endl;
                    }

                    cin.getline(car->model, 30);
                }
            }
        }
    }
}
```

```

    cout << "Year: ";
    while (!(cin >> car->year) || car->year < 1886 || car->year > 2026) {
        cout << "Wrong year. Enter year from 1886 to 2026" << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << "Price: ";
    while (!(cin >> car->price) || car->price <= 0) {
        cout << "Wrong price. Enter positive number: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Body: ";
    cin.getline(car->body, 20);

    while (cin.fail() || strlen(car->body) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Body car is too long. Maximum 19 characters" << endl;
        } else {
            cout << "Body car cannot be empty" << endl;
        }

        cin.getline(car->body, 20);
    }

    cout << "Segment: ";
    cin.getline(car->segment, 10);

    while (cin.fail() || strlen(car->segment) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Segment is too long. Maximum 9 characters" << endl;
        } else {
            cout << "Segment cannot be empty" << endl;
        }

        cin.getline(car->segment, 10);
    }

    ofstream file("cars.dat", ios::binary | ios::app);

    if (!file) {
        cout << "File open error" << endl;
    } else {
        file.write(reinterpret_cast<char*>(car), sizeof(Car));
        file.close();

        cout << "Car added." << endl;
    }

    break;
}

case 2: {
    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "Error file read" << endl;
    } else {
        cout << left
            << setw(31) << "Brand"
            << setw(31) << "Model"

```

```

        << setw(21) << "Body"
        << setw(11) << "Segment"
        << setw(8) << "Year"
        << setw(14) << "Price"
        << endl;

    cout << string(116, '-') << endl;

    while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
        cout << left
            << setw(31) << car->brand
            << setw(31) << car->model
            << setw(21) << car->body
            << setw(11) << car->segment
            << setw(8) << car->year
            << setw(14) << car->price
            << endl;
    }

    file.close();
}

break;
}

case 3: {
    int searchChoice;

    cout << "search by: " << endl;
    cout << "1 - brand" << endl;
    cout << "2 - model" << endl;
    cout << "3 - year" << endl;
    cout << "4 - price" << endl;
    cout << "5 - body" << endl;
    cout << "6 - segment " << endl;
    cout << "0 - back" << endl;

    while (!(cin >> searchChoice) || searchChoice < 0 || searchChoice > 6) {
        cout << "Wrong choice. Enter 0-6: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    if (searchChoice == 1) {
        char searchBrand[30];

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Enter Brand: ";
        cin.getline(searchBrand, 30);

        while (cin.fail() || strlen(searchBrand) == 0) {
            if (cin.fail()) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');

                cout << "Brand is too long. Maximum 29 characters" << endl;
            } else {
                cout << "Brand cannot be empty" << endl;
            }
        }

        cin.getline(searchBrand, 30);
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    } else {
        cout << left
            << setw(31) << "Brand"
            << setw(31) << "Model"
            << setw(21) << "Body"

```

```

        << setw(11) << "Segment"
        << setw(8) << "Year"
        << setw(14) << "Price"
        << endl;

    cout << string(116, '-') << endl;

    while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
        if (strstr(car->brand, searchBrand) != nullptr) {
            cout << left
                << setw(31) << car->brand
                << setw(31) << car->model
                << setw(21) << car->body
                << setw(11) << car->segment
                << setw(8) << car->year
                << setw(14) << car->price
                << endl;
        }
    }

    file.close();
}

if (searchChoice == 2) {
    char searchModel[30];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter model: " << endl;
    cin.getline(searchModel, 30);

    while (cin.fail() || strlen(searchModel) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Model is too long. Maximum characters 29" << endl;
        } else {
            cout << "Model cannot be empty" << endl;
        }
    }

    cin.getline(searchModel, 30);
}

ifstream file("cars.dat", ios::binary);

if (!file) {
    cout << "File open error" << endl;
} else {
    cout << left
        << setw(31) << "Brand"
        << setw(31) << "Model"
        << setw(21) << "Body"
        << setw(11) << "Segment"
        << setw(8) << "Year"
        << setw(14) << "Price"
        << endl;

    cout << string(116, '-') << endl;

    while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
        if (strstr(car->model, searchModel) != nullptr) {
            cout << left
                << setw(31) << car->brand
                << setw(31) << car->model
                << setw(21) << car->body
                << setw(11) << car->segment
                << setw(8) << car->year
                << setw(14) << car->price
                << endl;
        }
    }
}

```

```

        file.close();
    }
}

if (searchChoice == 3) {
    int minYear;
    int maxYear;

    cout << "From year: " << endl;

    while (!(cin >> minYear) || minYear < 1886 || minYear > 2026) {
        cout << "Wrong year. Enter year from 1886 to 2026" << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << "To year: " << endl;

    while (!(cin >> maxYear) || maxYear < 1886 ||
        maxYear > 2026 || maxYear < minYear) {
        cout << "Wrong year. Enter year from " << minYear << " to 2026: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    } else {
        cout << left
            << setw(31) << "Brand"
            << setw(31) << "Model"
            << setw(21) << "Body"
            << setw(11) << "Segment"
            << setw(8) << "Year"
            << setw(14) << "Price"
            << endl;

        cout << string(116, '-') << endl;

        while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
            if (car->year >= minYear && car->year <= maxYear) {
                cout << left
                    << setw(31) << car->brand
                    << setw(31) << car->model
                    << setw(21) << car->body
                    << setw(11) << car->segment
                    << setw(8) << car->year
                    << setw(14) << car->price
                    << endl;
            }
        }

        file.close();
    }
}

if (searchChoice == 4) {
    int minPrice;
    int maxPrice;

    cout << "From price" << endl;

    while (!(cin >> minPrice) || minPrice < 1) {
        cout << "Wrong price. Enter correct price." << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << "To price" << endl;
}

```

```

while (!(cin >> maxPrice) || maxPrice < 1 || maxPrice < minPrice) {
    cout << "Wrong price. Enter correct price." << endl;

    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

ifstream file("cars.dat", ios::binary);

if (!file) {
    cout << "File open error" << endl;
} else {
    cout << left
        << setw(31) << "Brand"
        << setw(31) << "Model"
        << setw(21) << "Body"
        << setw(11) << "Segment"
        << setw(8) << "Year"
        << setw(14) << "Price"
        << endl;

    cout << string(116, '-') << endl;

    while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
        if (car->price >= minPrice && car->price <= maxPrice) {
            cout << left
                << setw(31) << car->brand
                << setw(31) << car->model
                << setw(21) << car->body
                << setw(11) << car->segment
                << setw(8) << car->year
                << setw(14) << car->price
                << endl;
        }
    }

    file.close();
}

if (searchChoice == 5) {
    char searchBody[20];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter body: " << endl;
    cin.getline(searchBody, 20);

    while (cin.fail() || strlen(searchBody) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Body is too long. Maximum characters 19" << endl;
        } else {
            cout << "Body cannot be empty" << endl;
        }

        cin.getline(searchBody, 20);
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    } else {
        cout << left
            << setw(31) << "Brand"
            << setw(31) << "Model"
            << setw(21) << "Body"
            << setw(11) << "Segment"
            << setw(8) << "Year"
            << setw(14) << "Price"

```

```

        << endl;

        cout << string(116, '-') << endl;

        while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
            if (strstr(car->body, searchBody) != nullptr) {
                cout << left
                    << setw(31) << car->brand
                    << setw(31) << car->model
                    << setw(21) << car->body
                    << setw(11) << car->segment
                    << setw(8) << car->year
                    << setw(14) << car->price
                    << endl;
            }
        }

        file.close();
    }

    if (searchChoice == 6) {
        char searchSegment[10];

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Enter segment: " << endl;
        cin.getline(searchSegment, 10);

        while (cin.fail() || strlen(searchSegment) == 0) {
            if (cin.fail()) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');

                cout << "Segment is too long. Maximum characters 9" << endl;
            } else {
                cout << "Segment cannot be empty" << endl;
            }

            cin.getline(searchSegment, 10);
        }

        ifstream file("cars.dat", ios::binary);

        if (!file) {
            cout << "File open error" << endl;
        } else {
            cout << left
                << setw(31) << "Brand"
                << setw(31) << "Model"
                << setw(21) << "Body"
                << setw(11) << "Segment"
                << setw(8) << "Year"
                << setw(14) << "Price"
                << endl;

            cout << string(116, '-') << endl;

            while (file.read(reinterpret_cast<char*>(car), sizeof(Car))) {
                if (strstr(car->segment, searchSegment) != nullptr) {
                    cout << left
                        << setw(31) << car->brand
                        << setw(31) << car->model
                        << setw(21) << car->body
                        << setw(11) << car->segment
                        << setw(8) << car->year
                        << setw(14) << car->price
                        << endl;
                }
            }

            file.close();
        }
    }
}

```

```

        break;
    }

    case 0: {
        cout << "Exit" << endl;
        break;
    }

    default:
        cout << "Wrong choise" << endl;
        break;
    }

} while (choice != 0);

delete car;
return 0;
}

```


modular/main.cpp

```
#include <iostream>
#include <limits>

#include "functions.h"

using namespace std;

int main(){
    int choice;

    do {
        cout << "1 - Add car" << endl;
        cout << "2 - Show all" << endl;
        cout << "3 - Search" << endl;
        cout << "0 - Exit" << endl;

        while (!(cin >> choice) || choice < 0 || choice > 3) {
            cout << "Enter a number: ";

            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
        switch (choice) {
            case 1:
                addCar();
                break;

            case 2:
                showAll();
                break;

            case 3:
                findCar();
                break;

            case 0:
                cout << "Exit" << endl;
                break;

            default:
                cout << "Wrong choice" << endl;
                break;
        }
    } while (choice != 0);

    return 0;
}
```

modular/car.h

```
#ifndef CAR_H
#define CAR_H

struct Car {
    char brand[30];
    char model[30];
    int year;
    int price;
    char body[20];
    char segment[10];
};

#endif
```

modular/functions.h

```
#ifndef FUNCTIONS_H
#define FUNCTIONS_H

#include <iostream>
#include <fstream>
#include <cstring>
#include <limits>
#include <iomanip>
#include <string>

#include "car.h"

using namespace std;

void addCar();
void showAll();
void findCar();

void searchByBrand();
void searchByModel();
void searchByYear();
void searchByPrice();
void searchByBody();
void searchBySegment();

void printHeader();
void printCar(const Car& car);

void addCar() {
    Car car;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Brand: ";
    cin.getline(car.brand, 30);

    while (cin.fail() || strlen(car.brand) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Brand is too long. Maximum 29 characters: ";
        } else {
            cout << "Brand cannot be empty" << endl;
        }

        cin.getline(car.brand, 30);
    }

    cout << "Model: ";
    cin.getline(car.model, 30);

    while (cin.fail() || strlen(car.model) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Model is too long. Maximum 29 characters" << endl;
        } else {
            cout << "Model cannot be empty" << endl;
        }

        cin.getline(car.model, 30);
    }

    cout << "Year: ";

    while (!(cin >> car.year) || car.year < 1886 || car.year > 2026) {
```

```

        cout << "Wrong year. Enter year from 1886 to 2026" << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << "Price: ";

    while (!(cin >> car.price) || car.price <= 0) {
        cout << "Wrong price. Enter positive number: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Body: ";
    cin.getline(car.body, 20);

    while (cin.fail() || strlen(car.body) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Body car is too long. Maximum 19 characters" << endl;
        } else {
            cout << "Body car cannot be empty" << endl;
        }

        cin.getline(car.body, 20);
    }

    cout << "Segment: ";
    cin.getline(car.segment, 10);

    while (cin.fail() || strlen(car.segment) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Segment is too long. Maximum 9 characters" << endl;
        } else {
            cout << "Segment cannot be empty" << endl;
        }

        cin.getline(car.segment, 10);
    }

    ofstream file("cars.dat", ios::binary | ios::app);

    if (!file) {
        cout << "File open error" << endl;
    } else {
        file.write(reinterpret_cast<char*>(&car), sizeof(Car));
        file.close();

        cout << "Car added." << endl;
    }
}

void showAll() {
    Car car;

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "Error file read" << endl;
    } else {
        printHeader();

        while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
            printCar(car);
        }
    }
}

```

```

        file.close();
    }
}

void findCar() {
    int searchChoice;

    cout << "search by: " << endl;
    cout << "1 - brand" << endl;
    cout << "2 - model" << endl;
    cout << "3 - year" << endl;
    cout << "4 - price" << endl;
    cout << "5 - body" << endl;
    cout << "6 - segment " << endl;
    cout << "0 - back" << endl;

    while (!(cin >> searchChoice) || searchChoice < 0 || searchChoice > 6) {
        cout << "Wrong choice. Enter 0-6: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    switch (searchChoice) {
    case 1:
        searchByBrand();
        break;

    case 2:
        searchByModel();
        break;

    case 3:
        searchByYear();
        break;

    case 4:
        searchByPrice();
        break;

    case 5:
        searchByBody();
        break;

    case 6:
        searchBySegment();
        break;

    case 0:
        break;

    default:
        break;
    }
}

void searchByBrand() {
    Car car;
    char searchBrand[30];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter Brand: ";
    cin.getline(searchBrand, 30);

    while (cin.fail() || strlen(searchBrand) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Brand is too long. Maximum 29 characters" << endl;
        } else {
            cout << "Brand cannot be empty" << endl;
        }
    }
}

```

```

    }

    cin.getline(searchBrand, 30);
}

ifstream file("cars.dat", ios::binary);

if (!file) {
    cout << "File open error" << endl;
} else {
    printHeader();

    while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
        if (strstr(car.brand, searchBrand) != nullptr) {
            printCar(car);
        }
    }

    file.close();
}

}

void searchByModel() {
    Car car;
    char searchModel[30];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter model: " << endl;
    cin.getline(searchModel, 30);

    while (cin.fail() || strlen(searchModel) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Model is too long. Maximum characters 29" << endl;
        } else {
            cout << "Model cannot be empty" << endl;
        }

        cin.getline(searchModel, 30);
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    } else {
        printHeader();

        while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
            if (strstr(car.model, searchModel) != nullptr) {
                printCar(car);
            }
        }

        file.close();
    }
}

void searchByYear() {
    Car car;
    int minYear;
    int maxYear;

    cout << "From year: " << endl;

    while (!(cin >> minYear) || minYear < 1886 || minYear > 2026) {
        cout << "Wrong year. Enter year from 1886 to 2026" << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

```

```

cout << "To year: " << endl;

while (!(cin >> maxYear) || maxYear < 1886 ||
        maxYear > 2026 || maxYear < minYear) {
    cout << "Wrong year. Enter year from " << minYear << " to 2026: ";

    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

ifstream file("cars.dat", ios::binary);

if (!file) {
    cout << "File open error" << endl;
} else {
    printHeader();

    while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
        if (car.year >= minYear && car.year <= maxYear) {
            printCar(car);
        }
    }

    file.close();
}

void searchByPrice() {
    Car car;
    int minPrice;
    int maxPrice;

    cout << "From price" << endl;

    while (!(cin >> minPrice) || minPrice < 1) {
        cout << "Wrong price. Enter correct price." << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << "To price" << endl;

    while (!(cin >> maxPrice) || maxPrice < 1 || maxPrice < minPrice) {
        cout << "Wrong price. Enter correct price." << endl;

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    } else {
        printHeader();

        while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
            if (car.price >= minPrice && car.price <= maxPrice) {
                printCar(car);
            }
        }

        file.close();
    }
}

void searchByBody() {
    Car car;
    char searchBody[20];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

```

```

cout << "Enter body: " << endl;
cin.getline(searchBody, 20);

while (cin.fail() || strlen(searchBody) == 0) {
    if (cin.fail()) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Body is too long. Maximum characters 19" << endl;
    } else {
        cout << "Body cannot be empty" << endl;
    }

    cin.getline(searchBody, 20);
}

ifstream file("cars.dat", ios::binary);

if (!file) {
    cout << "File open error" << endl;
} else {
    printHeader();

    while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
        if (strstr(car.body, searchBody) != nullptr) {
            printCar(car);
        }
    }

    file.close();
}

void searchBySegment() {
    Car car;
    char searchSegment[10];

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter segment: " << endl;
    cin.getline(searchSegment, 10);

    while (cin.fail() || strlen(searchSegment) == 0) {
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Segment is too long. Maximum characters 9" << endl;
        } else {
            cout << "Segment cannot be empty" << endl;
        }

        cin.getline(searchSegment, 10);
    }

    ifstream file("cars.dat", ios::binary);

    if (!file) {
        cout << "File open error" << endl;
    } else {
        printHeader();

        while (file.read(reinterpret_cast<char*>(&car), sizeof(Car))) {
            if (strstr(car.segment, searchSegment) != nullptr) {
                printCar(car);
            }
        }

        file.close();
    }
}

void printHeader() {
    cout << left

```



```

        << setw(31) << "Brand"
        << setw(31) << "Model"
        << setw(21) << "Body"
        << setw(11) << "Segment"
        << setw(8) << "Year"
        << setw(14) << "Price"
        << endl;

    cout << string(116, '-') << endl;
}

void printCar(const Car& car) {
    cout << left
        << setw(31) << car.brand
        << setw(31) << car.model
        << setw(21) << car.body
        << setw(11) << car.segment
        << setw(8) << car.year
        << setw(14) << car.price
        << endl;
}

#endif

```